

计算机组成原理

学号：2110853 姓名：朱子轩

一、系统设计（总体设计）

目标：性能评价（关键问题1）

个人计算机更看重响应时间，服务器更看重吞吐率。我们主要考虑响应时间，将性能定义为

$$\text{性能} = 1/\text{执行时间}$$

时间

我们可以用如下方法计算CPU执行时间

$$\begin{aligned}\text{CPU执行时间} &= \text{CPU时钟周期数} \times \text{时钟周期时间} \\ &= \text{CPU时钟周期数} / \text{时钟频率}\end{aligned}$$

而CPU时钟周期数可以写成

$$\text{CPU时钟周期数} = \text{程序的指令数} \times \text{每条指令的平均时钟周期数}$$

我们定义

- CPI：每条指令的时钟周期数

于是可以得到经典的CPU性能公式

$$\text{CPU时间} = \text{指令数} \times \text{CPI} \times \text{时钟周期时间}$$

或

$$\text{CPU时间} = \text{指令数} \times \text{CPI} / \text{时钟频率}$$

注意CPI是一个平均值，具体的每条指令的时钟周期数可能不一样，如果要计算总的CPI可能需要加权平均
不同指标对CPU时间的影响

硬件或软件指标	影响什么
算法	指令数，可能的CPI
编程语言	指令数，CPI
编译程序	指令数，CPI
指令集体系结构	指令数，CPI，时钟频率

还有一个指标MIPS，每秒执行的百万指令数

$$\text{MIPS} = \text{指令数} / (\text{执行时间} \times 10^6) = \text{时钟频率} / (\text{CPI} \times 10^6)$$

但是这个度量存在问题，详见[谬论与陷阱](#)。

改进后的程序执行时间可以使用**Amdahl定律**计算：

$$\text{改进后的执行时间} = \text{受改进影响的执行时间} / \text{改进量} + \text{不受影响的执行时间}$$

例如，假设一个程序在一台计算机上运行需要100秒，其中80秒用于乘法操作，如果想把该程序的运行速度提高到4倍，乘法操作的速度需要改进多少？

在这里，改进后的执行时间为25秒，受改进影响的执行时间为80秒，不受影响的执行时间为20秒，那么改进量为 $80 / (25 - 20) = 16$ ，即乘法操作的速度需要是原来的16倍。

空间

空间方面就是吞吐率的影响。吞吐率也叫带宽，表示单位时间完成的任务数量。

困境：功率墙（关键问题2）

随着CPU集成度越来越高，晶体管数量剧增，功耗越来越大，散热越来越难。

占统治地位的集成电路技术是CMOS（互补型金属氧化物半导体），其功耗主要来源于动态能耗，即在晶体管开关过程中产生的能耗，即晶体管从0翻转到1或从1翻转到0消耗的能量。

每个晶体管的功耗

$$\text{功耗} \propto 1/2 \times \text{负载电容} \times \text{电压}^2 \times \text{开关频率}$$

其中开关频率是时钟频率的函数。

功率墙的问题，就是我们无法再通过降低电压来降低功率（因为电压已经从5V降低到了1V，无法继续降低），也无法避免产生更多的热能。

为了解决功率墙的问题，有以下几个方向

- 新组合：从单核处理器转变为多核处理器
- 新材料：如光纤，超导计算机，碳基芯片
- 新架构：类脑计算
- 新机理：量子计算机？

原则：八大思想（关键问题3）

八大思想分为两个设计原则，4个提高性能的方法，2个存储器层次与冗余思想。具体如下

1. 面向摩尔定理设计
2. 使用抽象简化设计
3. 加速大概率事件
4. 通过并行提高性能
5. 通过流水线提高性能
6. 通过预测提高性能
7. 存储器层次
8. 通过冗余提高可靠性

切入点

构建式

从无到有建立一个新系统，关键在确定**流程**。

改良式

改进现有的系统，关键在找出改进的**关键**。

二、指令设计（类比设计）

路线选择：RISC和CISC（关键问题4）

CISC

CISC(Complex Instruction Set Computer)，即“复杂指令系统计算机”，简言之，为新增的功能来设计新的指令。

主要包括x86，占据主要的PC和服务市场。

RISC

RISC(Reduced Instruction Set Computer)，即“精简指令集计算机”，简言之，用少数指令的组合来实现新增的功能。

主要包括

- ARM：以智能手机为代表的嵌入式市场
- MIPS：以网络设备为代表的嵌入式市场
- PowerPC：IBM维护的服务器/游戏主机
- RISC-V：冉冉升起的新兴开源指令集

优点：

- 高频指令：加速大概率事件
- 等长规整：为了并行而加速，执行效率高
- 积木模式：减少指令实现的复杂程度

缺点：

- 兼容性较差：版本之间兼容性较差
- 代码规模较大：程序设计相对不友好，严重依赖编译器

目标

功能：运算

一个指令集首先要支持各种运算。

改进：流水线

我们需要让计算机运算更快，或者成本更低。

原则：指令设计原则（关键问题5）

- 简单源于规整
- 越小越快
- 加速大概率事件
- 优秀的设计需要适宜的折中方案

指令集设计

具体 → 一般 → 迭代验证 → 泛化验证与迭代

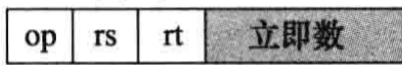
指令间设计

单条指令 → 多条指令

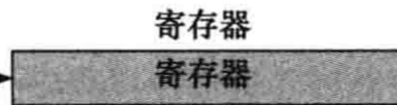
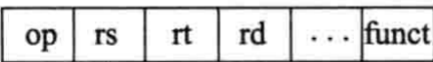
形成两个问题：数据寻址、地址寻址

寻址方式（关键问题6）

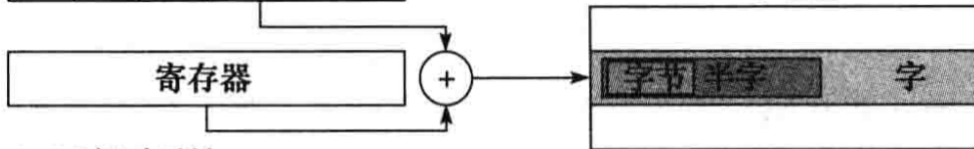
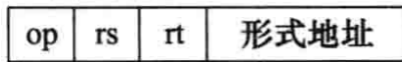
1. 立即数寻址



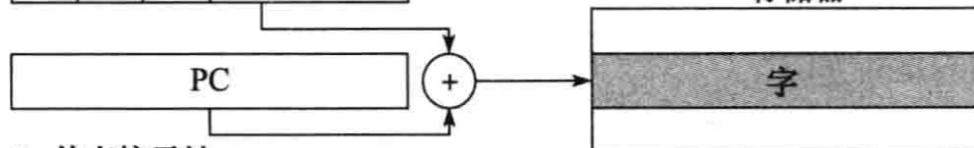
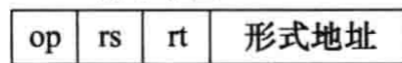
2. 寄存器寻址



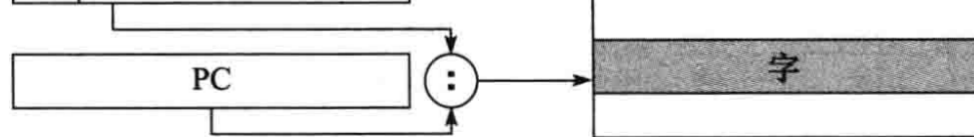
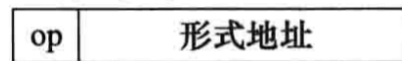
3. 基址寻址



4. PC相对寻址



5. 伪直接寻址



下面总结五种寻址方式。

立即数寻址。

op(6位)	rs(5位)	rt(5位)	立即数(16位)
--------	--------	--------	----------

操作数是位于指令自身中的常数。

寄存器寻址。

op(6位)	rs(5位)	rt(5位)	rd(5位)	shamt(5位)	funct(6位)
--------	--------	--------	--------	-----------	-----------

操作数是rs代表的寄存器。

基址寻址（偏移寻址）。

op(6位)	rs(5位)	rt(5位)	形式地址(16位)
--------	--------	--------	-----------

操作数在内存中，其地址是指令中基址寄存器和常数的和。

PC相对寻址。

op(6位)	rs(5位)	rt(5位)	形式地址(16位)
--------	--------	--------	-----------

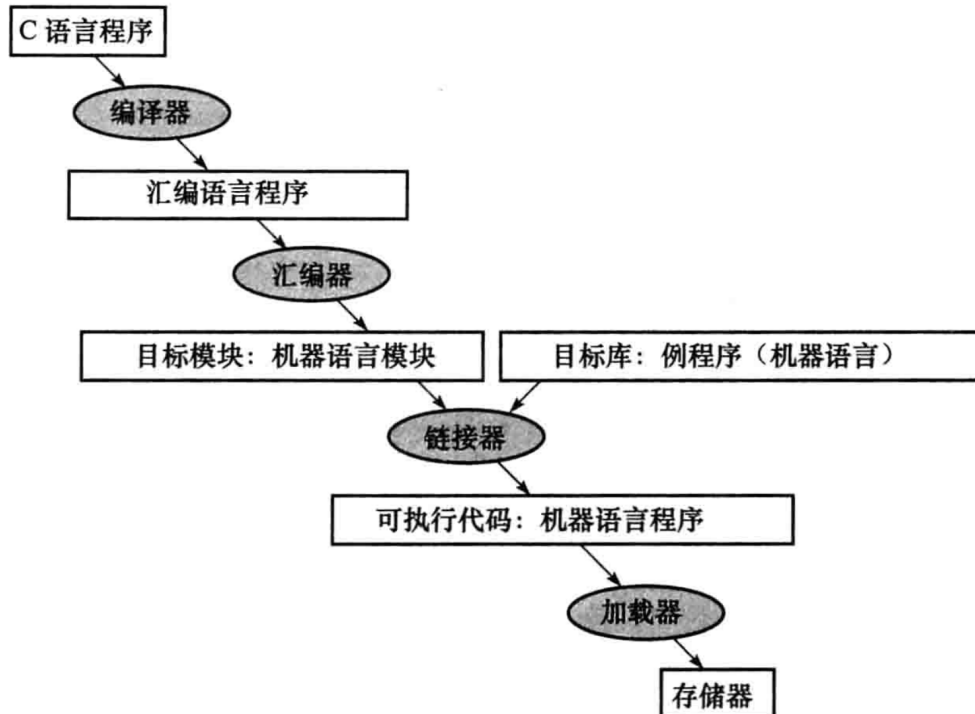
操作数在内存中。将16位地址左移2位与PC相加，得到操作数的地址。注意此时的PC已经加上了4。

伪直接寻址。

op(6位)	形式地址(26位)
--------	-----------

操作数在内存中。将26位地址左移2位与PC计数器的高4位相连，得到操作数的地址。

程序：编译与链接（关键问题7）



编译器：编译器将C程序转换成一种机器能理解的符号形式的**汇编语言程序**。

汇编器：汇编器能够处理一些机器语言的变种（伪指令），硬件不需要实现这些指令，但是他们在汇编语言中的存在简化了程序转换和编程。汇编器将汇编语言程序转换成目标文件，它包括机器语言指令、数据和指令正确放入内存所需要的信息。

为了产生汇编语言程序中每条指令对应的二进制表示，汇编器必须处理所有标号对应的地址。汇编器将分支和数据传输指令中用到的标号都放入一个**符号表**中，这个表由标号和地址成对构成。

链接器：它是一个系统程序，把各个独立汇编的机器语言程序组合起来并且解决所有未定义的标记，最后生成可执行文件。

链接器的工作分为三个步骤：

1. 将代码和数据模块象征性地放入内存。
2. 决定数据和指令标签的地址。
3. 修补内部和外部的引用。

链接器使用每个目标模块中的重定位信息和符号表，来解析所有未定义标签。这种引用发生在分支指令、跳转指令和数据寻址处。他寻找所有的旧地址并用新地址取代他们。采用链接器的原因是修补代码比重新编译汇编要快得多。

如果所有的外部引用都解析完，链接器接着决定每个模块将要占用的内存位置。因为文件是单独汇编的，所以汇编器不可能知道该模块的指令和数据相对于其他模块而言将会被放到哪里。当链接器将一个模块放到内存中的时候，所有的绝对引用，即与寄存器无关的内存地址必须重定位以反映它的真实地址。

链接器产生一个可执行文件，它可以在一台计算机上运行。童虫，这个文件与目标文件既有相同的格式，但是它不包含未解决的引用。

加载器：把目标程序装载到内存中以准备运行的系统程序。

在UNIX系统中，加载器按照如下步骤工作：

1. 读取可执行文件头来确定代码段和数据段的大小。
2. 为正文和数据创建一个足够大的地址空间。
3. 将可执行文件中的指令和数据复制到内存中。
4. 把主程序的参数（如果存在）复制到栈顶。
5. 初始化机器寄存器，将栈指针指向第一个空位置。
6. 跳转到启动例程，它将参数复制到参数寄存器并调用程序的main函数。当main函数返回时，启动例程通过系统调用exit终止程序。

三、部件设计（类比设计）

自然流程

整数，浮点数的加减法和乘法

数字化自然流程

优化

降本

- 压缩寄存器：从64位寄存器换成32位寄存器
- 复用寄存器
- 减少位宽

增效

加法：超前进位、分组进位

乘法：快速乘法器、进位保留加法器

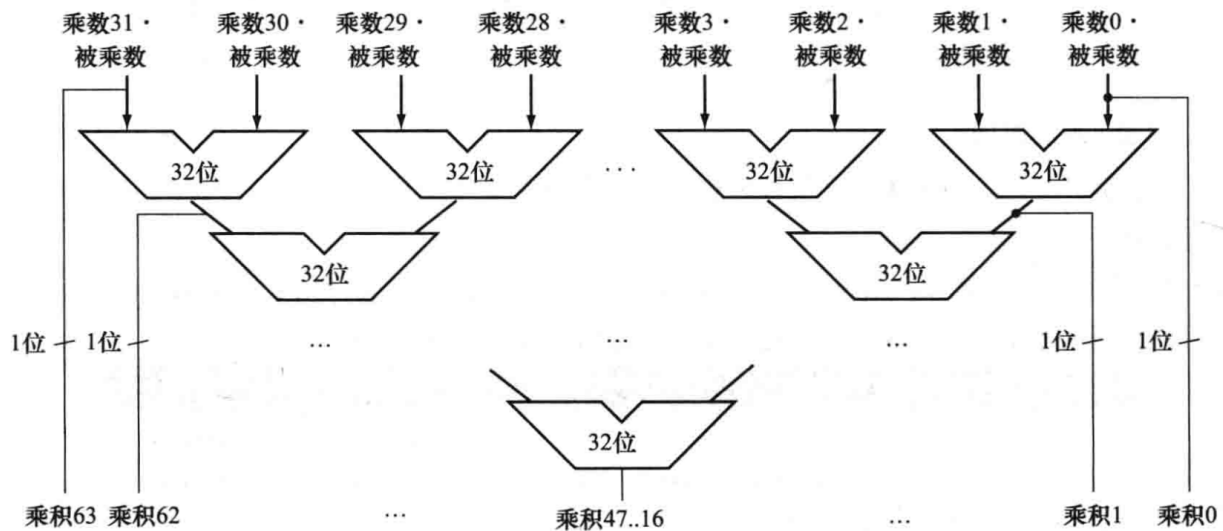


图 3-7 快速乘法器硬件结构。这个结构使用 31 个加法器“展开循环”来实现最小的时延，而不再是使用单个 32 位的加法器 31 次

浮点数（关键问题8）

IEEE 754

单精度浮点数，占用1个32位的字

s(1位)	指数(8位)	尾数(23位)
-------	--------	---------

单精度浮点数的偏阶是127，即 $2^7 - 1$ ，于是单精度浮点数在形式上可以表示成

$$(-1)^s + 1.\text{尾数} \times 2^{\text{指数}-127}$$

双精度浮点数，占用2个32位的字

s(1位)	指数(11位)	尾数(52位)
-------	---------	---------

双精度浮点数的偏阶是1023，即 $2^{10} - 1$ ，于是单精度浮点数在形式上可以表示成

$$(-1)^s + 1.\text{尾数} \times 2^{\text{指数}-1023}$$

例如使用单精度和双精度分别表示数 -0.75_{10} ，先将其转换为二进制表示，即

$$-0.75_{10} = -3/4_{10} = -3/2^2_{10} = -11_2/2^2_{10} = -0.11_2$$

用科学计数法表示为

$$-1.1_2 \times 2^{-1}$$

于是单精度浮点数中，符号位为1，尾数为1和22个0，指数为 $-1 + 127 = 126 = 01111110_2$ ，存储为

1011 1111 0100 0000 0000 0000 0000 0000

双精度浮点数中，符号位为1，尾数为1和51个0，指数为 $-1 + 1023 = 1022 = 011111111110_2$ ，存储为

1011 1111 1110 1000 0000 0000 0000 0000 0000 0000 0000 0000 0000 0000 0000 0000

下面讨论浮点数的表示范围。

规格数就是尾数位隐藏的整数部分为1，非规格数就是尾数位隐藏的整数部分为0。

如果指数位全为0，代表存储的数是一个非规格数，如果指数位为全1，代表存储的数是一个特殊数，比如正负无穷，或者NaN。

所以，以单精度浮点数为例，指数部分存储的有效值区间为[0000 0001, 1111 1110]，即[1, 254]，减去偏阶127后实际的指数为[-126, 127]

而尾数的存储范围为全0到全1，可以看成是[0, 1)，于是单精度浮点数的表示范围可以近似表示为

$$(-2 \times 2^{127}, -1 \times 2^{-126}] \cup [1 \times 2^{-126}, 2 \times 2^{127})$$

工程价值

精度问题：IEEE754在中间计算时，总是多保留两位，分别称为保护位（guard）和舍入位（round），具体误差不超过0.5个ulp（尾数最低位）。

浮点数运算不满足结合律。

四、数字通路设计（常规改进设计与类比设计）

常规方法

建立流程→优化流程→迭代解决问题

单周期计算机的建立

时钟周期（关键问题9）

补充： $1ns = 10^{-9}s$ ， $1ps = 10^{-12}s$

需要实现的指令：

- R型指令：add, sub, and, or, slt
- 访存指令：lw, sw
- 决策指令：beq

具体数据通路如下

- R型指令（加减法）—— `add $t1, $t2, $t3`
 1. 从指令存储器中取出指令，PC自增。
 2. 从寄存器堆中读出两个寄存器的值。同时，主控制单元计算出各控制信号的状态。
 3. ALU根据funct字段确定ALU功能，对从寄存器堆读出的数据进行操作。
 4. 将ALU的结果写回寄存器堆，根据指令的[15:11]选择目标寄存器。
- 取数指令 —— `lw $t1, offset($t2)`
 1. 从指令存储器中取出指令，PC自增。
 2. 从寄存器堆中读出寄存器\$t2 的值。

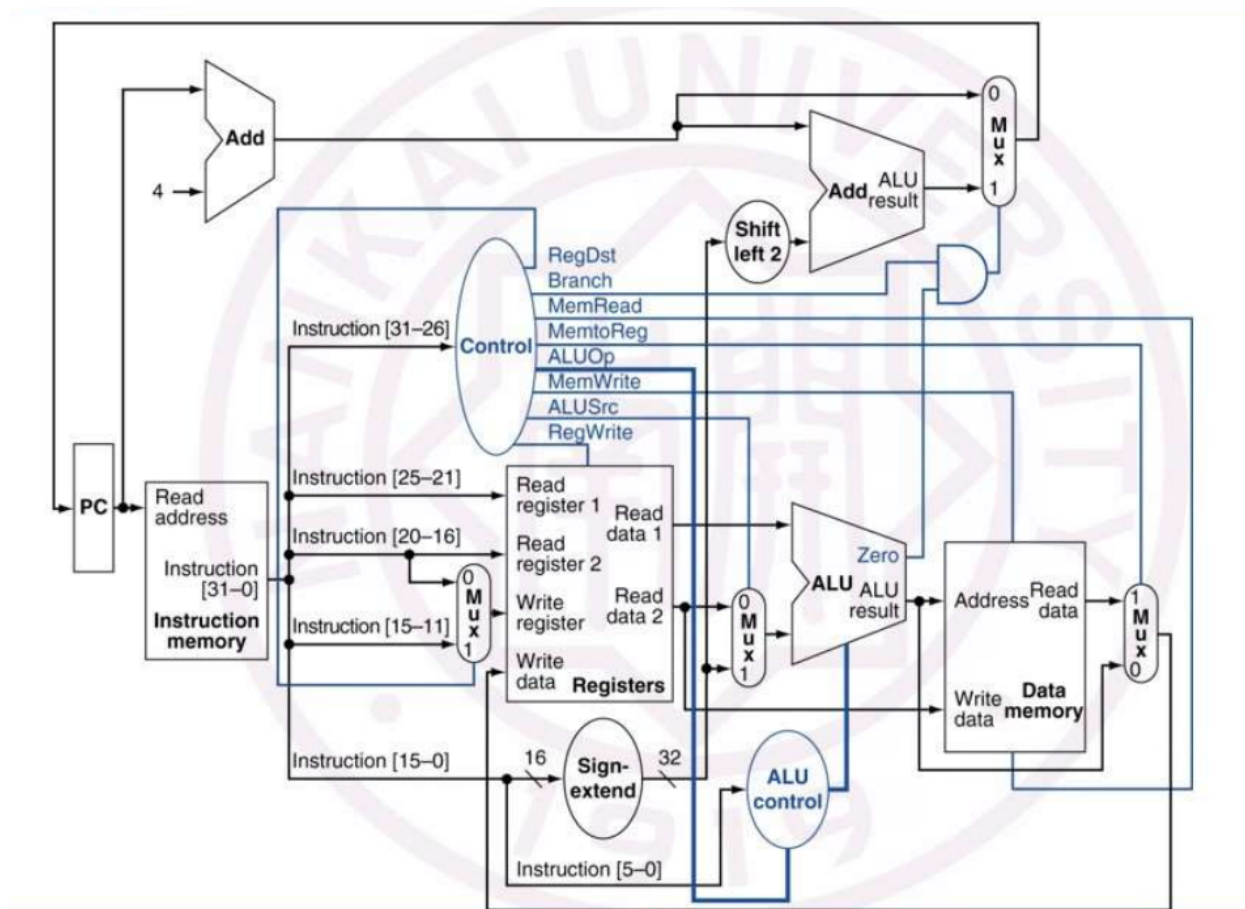
3. ALU从寄存器堆读出的值与符号扩展后的指令低16位相加。
4. 将ALU的结果作为数据存储器的地址。
5. 存储单元的数据写回寄存器堆，根据指令的 [20:16] 选择目标寄存器。

• beq指令 —— `beq $t1 $t2 offset`

1. 从指令存储器中取出指令，PC自增。
2. 从寄存器堆中读出寄存器\$t1、\$t2 的值。
3. ALU将寄存器堆读出的两个数相减。PC+4的值与符号扩展并左移2位后的指令低16位相加，结果即分支目标地址。
4. 根据ALU的零输出决定哪个加法器的结果存入PC中

在单周期中，时钟周期对所有指令等长，因此由执行最慢的指令决定，即lw指令。这样虽然CPI为1，但是时钟周期太长，执行速度还是太慢。

数据通路图



多周期计算机（类比设计）

评价：流水线的计算与设计（关键问题10）

一条MIPS指令的执行可以分为五个阶段：

1. IF取指令：根据PC所给的地址，从存储器里取出指令
2. ID译码与读寄存器：分析指令字段，读取一个或两个寄存器
3. EX运算：ALU运算R型指令的结果/访存指令的地址/beq两源操作数是否相等

4. MEM访存与分支：访存指令向存储器进行读写，分支指令完成分支

5. WB写回：将结果送回某寄存器

	IF取指令	ID译码与读reg	EX运算	MEM访存分支	WB写回
R型	根据PC 从存储器中 取出指令	取源操作数rs和rt	执行运算	-	将ALU运算结果写回rd
lw		取基址寄存器rs 偏移量符号扩展	计算内存地址	读数据存储器 取得数据	将访存数据写回rt
sw				写数据存储器	-
beq		取rs和rt 字地址符号扩展 并左移两位	比较rs和rt 算分支目标地址	地址写回PC	-

在相邻两级流水线之间加入寄存器，用于传输数据和存储中间计算结果。

所有的流水级都花费一个时钟周期时间因此，时钟周期必须能够满足最慢操作的执行需要。

流水线所带来的性能提高是通过增加指令的吞吐率，而不是减少单条指令的执行时间实现的。

困境

三种阻塞与解决办法（关键问题11）

结构冒险：硬件不支持多条指令在同一时钟周期执行。

解决办法：拆成指令和数据缓存。

数据冒险：无法提供指令所需数据导致指令不能在预定的时钟周期内执行。

比如alu运算结果在EX段之后就能得到，但是在WB阶段才能写回寄存器，但是下一条指令需要在EX段就使用该数据。

解决办法：

- EX-EX旁路。将ALU的输出作为ALU的输入。
- MEM-EX旁路。在访存阶段之后，将ALU的输出作为ALU的rt输入。两种旁路都有的称为全旁路。
- 气泡法。当出现**取数-使用型数据冒险**时，无法通过旁路解决，所以在执行第一条指令后加入一个阻塞周期，形成流水线阻塞。当然也可以通过指令重排序来避免这种数据冒险。

控制冒险：由于指令地址变化不是流水线所预期的，导致不能在预定的时钟内执行。

解决办法：

- 提前计算分支结果：在ID级增加额外的电路计算分支目标地址、判断分支条件。但是这种方法并不总是有用，会受到数据依赖的限制。

当采取这个办法时，那么如果前一条指令是lw，需要阻塞两个时钟周期；如果需要使用上一条的计算数据，需要阻塞1个时钟周期。具体体现在书后习题4.14中。

- 静态预测：总是假设分支不执行，继续执行紧跟在beq后指令。
- 动态预测：保存每次分支的历史记录，利用历史记录预测。

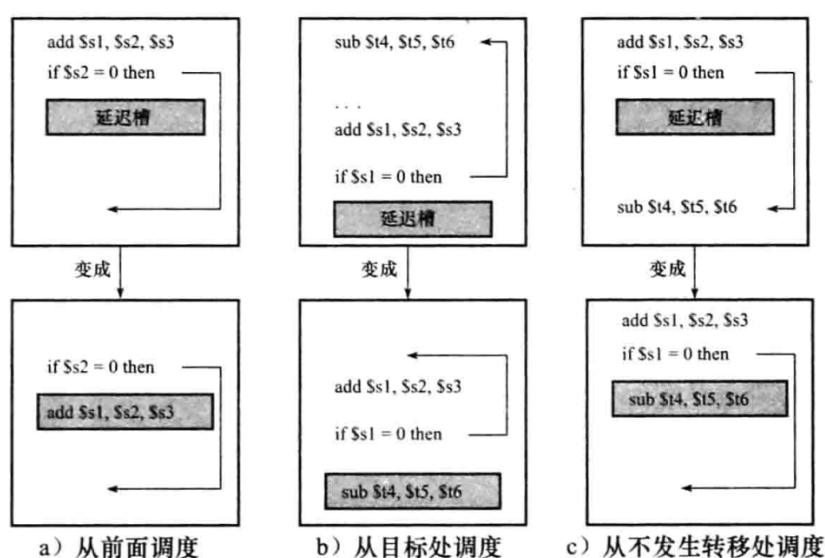
分支延迟时间槽，一般就是将分支后的一条语句加入分支延迟时间槽中，在分支指令之后执行。

编译器和汇编器都会试图把总在分支后执行的那条指令放入分支延迟时间槽（branch delay slot）。这些软件的作用就是使后续的指令有效并且有用。图 4-64 给出了三种调度分支延迟时间槽的方法。

延迟分支调度的限制在于：对能够被调度到分支延迟时间槽中的指令的限制；在编译时对分支发生与否的预测能力。

对每个时钟周期发射一条指令的五级流水线处理器而言，延迟分支是一种简单有效的方法。随着处理器向更深流水线以及单周期多指令的方向发展（见 4.10 节），分支延迟变得更长，单延迟时间片实际上并没有多大作用。所以，与开销大但更灵活的动态预测方法相比，延迟分支技术已经失去了吸引力。同时，根据摩尔定律使动态预测的成本相对更低。

分支延迟时间槽：紧跟延迟分支指令的时间片。在 MIPS 体系结构中，用不影响分支的一条指令填充到该时间片中。



- 64 分支延迟时间槽的调度。每一对方框中的上面一个表示调度前的代码，下面一个表示调度后的代码。在方案 a 中，延迟时间片通过插入分支之前的一条与分支无关的指令实现，这是一种最佳的选择。当方案 a 无法实现时，就使用方案 b 和方案 c。在方案 b 和方案 c 的代码序列中，分支条件中使用了 \$1，因而不能将 add 指令（它的目的寄存器是 \$1）移入分支延迟时间槽。方案 b 中的分支延迟时间槽是按照分支目标地址调度的。由于目标指令可以通过其他路径访问到，通常需要将它们进行复制。当分支发生的可能性比较大时，一般选择方案 b，如循环分支。最后，也可能采用方案 c 预测分支不发生的一条指令进行调度。为了使方案 b 和方案 c 中的优化合法，sub 指令必须在分支预测错误时也能“正常”执行。“正常”意味着虽然有些工作是多余的，但程序依然能够正确执行。例如，当分支预测错误且 \$t4 是未被使用的临时寄存器时，就是这种情况

异常与中断（关键问题12）

异常：指打断程序正常执行的突发事件，用于检测溢出等。

中断：来自处理器外部的异常。

常见的异常：算术溢出、使用未定义的指令、用户程序进行操作系统调用、硬件故障

常见的中断：I/O设备请求、硬件故障

为了处理异常，需要增加两个寄存器：EPC和Cause

当异常发生时，处理器要在异常程序计数器(EPC)中保存出错指令的地址，并把控制权转交给操作系统的特定地址。操作系统可以采取适当的行动，在处理完异常之后可以终止程序也可以继续执行程序。

为了处理异常，操作系统除了要知道是那条质量引起的异常之外，还必须直到引起异常的原因。主要有两种方式用于表示产生异常的原因。MIPS使用的方法是设置一个状态寄存器（称为Cause寄存器），其中有一个字段用于记录异常产生的原因。

还可以采取向量中断。在向量中断中，控制权被转移到由异常原因决定的地址处。操作系统根据引起异常的地址得知导致异常的原因，并作相应的处理。

优化

流水线优化方向（关键问题13）

指令级并行：增加流水线深度以重叠更多的指令（超流水）；复制计算机内部部件数量，使得每个流水级可以启动多条指令（多发射）。

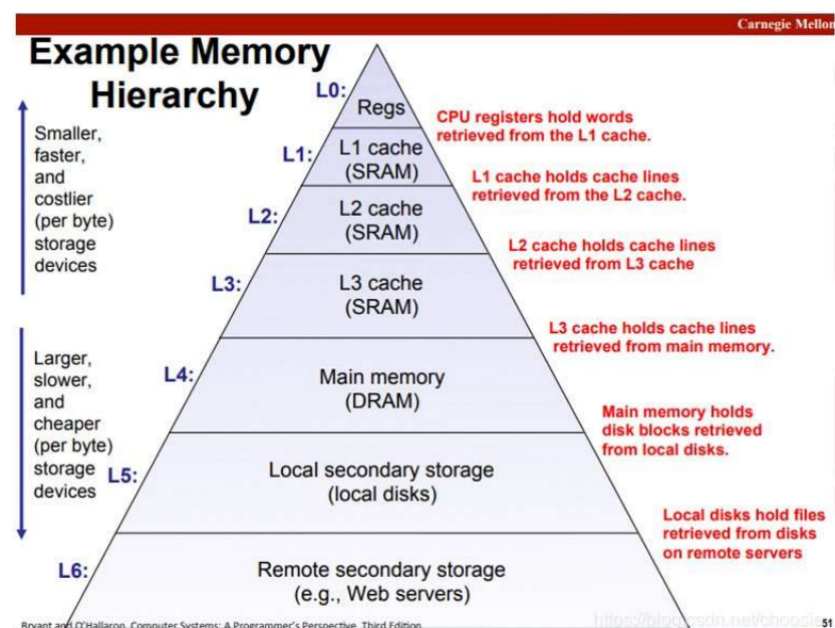
超长指令字：一类可以同时启动多个操作的指令集，其中操作在单个指令中相互独立，并且一般都有独立的操作码字段。是由编译器实现的静态多发射。

超标量：CPU内有多条流水线，这些流水线能够并行处理。在单流水线结构中，指令虽然能够重叠执行，但仍然是顺序的，每个周期只能发射或回收一条指令。超级标量结构的CPU支持指令级并行，每个周期可以发射多条指令(2-4条居多)。这样,可以使得CPU的IPC > 1, 从而提高CPU处理速度。是由硬件实现的动态多发射。

五、存储设计（组合设计）

存储器层次（关键问题14）

从上往下，容量越来越大，速度越来越慢。



- L0：寄存器
- L1~L3：高速缓存cache，集成在CPU里，采用静态随机访问存储器（SRAM）
- L4：内存，采用动态随机访问存储器（DRAM）

- L5: 二级存储器或辅存, 比如磁盘, 闪存
- L6: 远程的存储, 比如网络服务

SRAM、DRAM在断电后很快丢失数据, 称为易失性存储器; 磁盘和闪存断电后不丢失数据, 称为非易失性存储器。

DRAM需要周期性进行刷新, 才能保持数据。

在上述结构中, 除非数据在 $i + 1$ 层存在, 否则绝不可能在第 i 层存在。

程序访问的局部性原理 (关键问题15)

时间局部性: 如果某个数据项被访问, 那么在不久的将来他可能再次被访问。

空间局部性: 如果某个数据项被访问, 与他地址相邻的项可能很快也将被访问。

Cache (关键问题16)

单位换算: $1KiB = 8Kib$, $1KiB = 2^{10}B = 1024$ 字节, $1字 = 4$ 字节

建立

直接映射: 根据字的块地址映射到cache中的块, 映射方法为

$$(\text{块地址}) \bmod (\text{cache中的块数})$$

在直接映射里, 每个存储器地址仅仅对应到cache中的一个位置。如果cache的块数为2的幂个, 那么上述映射方法就相当于取块地址的低几位。

对于 2^n 块的cache, cache块号=块地址的后 n 位。

由于cache中每个位置可能对应于主存中多个不同的地址, 为了确定cache中的数据项是否是我们请求的字, 可以在cache中增加一组**标记**(tag), 标记中包含了地址信息, 标记只需要包含地址的高位。

有的时候标记域中的值没有意义, 比如处理器启动时cache中没有数据。所以还需要增加一个**有效位**(valid bit), 标识一个块是否含有一个有效地址。

前面的计算公式是对于cache中每一块大小都是1个字节得到的。但是实际的cache中, 由于MIPS指令是按字(4字节)对齐的, 并且cache中的每一块可能有多个字, 这时的映射的计算方式就需要调整, 需要先将字节地址转换为块地址: $\text{块地址} = \text{字节地址} / \text{每块字节数}$ 。

注意看题目中给的是字数还是字节数。

假设块地址有32位, 将其存入cache中, 假设cache的块数为 2^n , 每块的大小为 2^m 个字(2^{m+2} 个字节), 从高位到低位会依次被分为以下部分:

- $32 - (n + m + 2)$ 位, 存入标记域
- n 位, 等于cache块的地址
- m 位, 可以用于查找块中的字
- 2位, 可以作为字节偏移信息

例: 假设一个直接映射的cache, 有16KiB的数据, 块大小为4个字, 地址为32位, 那么该cache总共需要多少位?

我们知道16KiB是4096个字，块的大小为4个字(2^2)，那么就有1024(2^{10})个块。每个块有 4×32 即128位数据，加上 $32 - 10 - 2 - 2$ 位的标记域，再加上1位的有效位，因此总的cache大小为

$$2^{10} \times (4 \times 32 + (32 - 10 - 2 - 2) + 1) = 2^{10} \times 147 = 147\text{Kib} = 18.4\text{KiB}$$

即能装16KiB数据的cache总共需要18.4KiB的容量。

例：考虑一个cache中有64个块，每块大小为16字节，那么字节地址1200将被映射到cache中哪一块？

首先计算块地址

$$\text{块地址} = \text{字节地址} / \text{每块的字节数} = 1200 / 16 = 75$$

然后计算cache块

$$(\text{块地址}) \bmod (\text{cache中的块数}) = 75 \bmod 64 = 11$$

所以对应的cache中的块号为11。事实上，地址1200和1215之间的所有地址都映射在这一块。

当发生cache缺失，就需要替换一个位置的块。在直接映射中，需要替换原来那个位置的块。在组相联的块中，可以替换对应的组中的一块。在全相联的块中，所有的块都可以被替换。块替换块的选择，有两种策略：

- 最近最少使用(LRU)法：替换最久没有使用的块
- 先进先出(FIFO)法：替换最早加入cache的块

优化

全相联：一个块可以放置到cache中的任何位置。

组相联：一个块可以放到cache中的部分位置（至少两个）。

组相联的影身公式为 $\text{cache组号} = \text{内存块号} \bmod \text{cache组数}$ 。

01 例题·标记位大小与组相联

提高相联度需要更多比较器，同时 cache 块中的标记位数也需要增加。假设一个 cache 有 4 096 个块，块大小为 4 个字，地址为 32 位，请分别计算在直接映射、两路组相联、四路组相联和全相联映射中，cache 的总组数以及总的标记位数。

01 答案

由于块大小为 16 ($=2^4$) 字节，32 位地址域中的 $32 - 4 = 28$ 位用来提供索引和标记位。直接映射中组数和块数一样，由于 $\log_2(4\,096) = 12$ ，因此有 12 位是索引位；因此总的标记位数是 $(28 - 12) \times 4\,096 = 16 \times 4\,096 = 66\text{Kb}$ 。

相联度每增加 1 倍，组数就会减少 1/2，因此用来索引 cache 的位数也要相应减 1，而标记位则是增 1。因此，对于一个两路组相联 cache，有 2 048 个组，总的标记位数为 $(28 - 11) \times 2 \times 2\,048 = 34 \times 2\,048 = 70\text{Kb}$ 。而四路组相联中组数为 1 024，那么总的标记位数为 $(28 - 10) \times 4 \times 1\,024 = 72 \times 1\,024 = 74\text{Kb}$ 。

对于全相联 cache，只有一个有 4 096 个块的组，标记位是 28 位，因此总的标记位数是 $28 \times 4\,096 \times 1 = 115\text{Kb}$ 。 □

组合：多级cache

一级cache：致力于减少命中时间获得较短的时钟周期或者较少的流水级。

二级cache：针对改善缺失率以减少长时间的访存代价。

系统内部主存

逻辑结构：TLB-Cache-虚存-磁盘

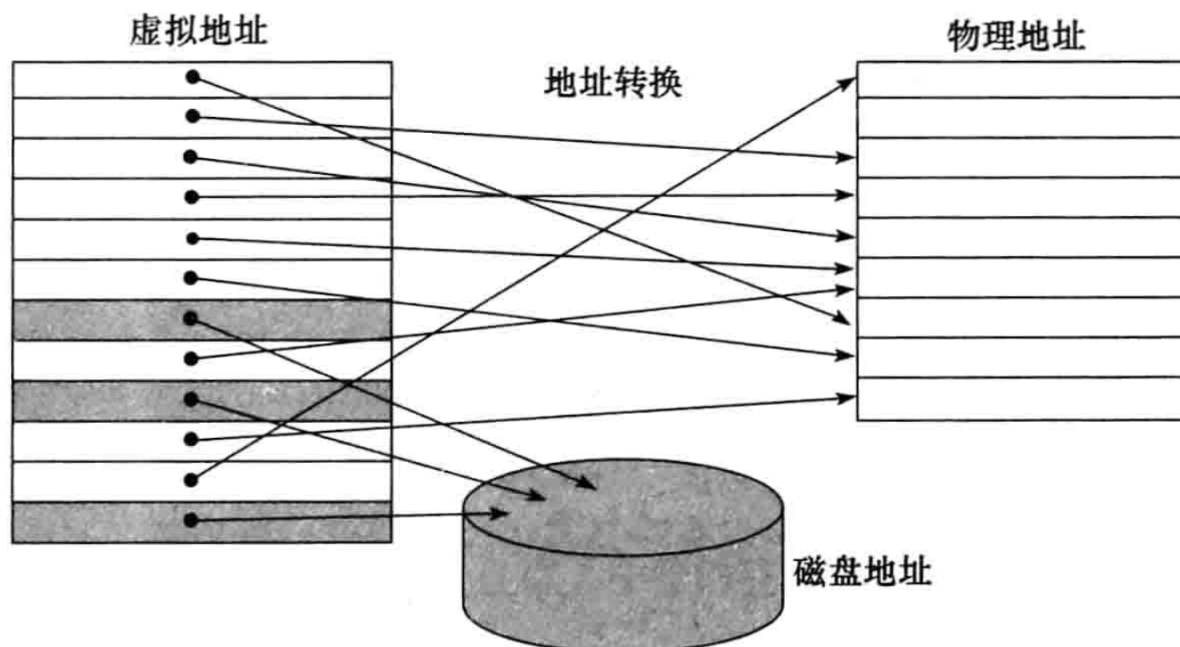
虚存（关键问题17）

虚拟存储器：一种将主存用作辅助存储器高速缓存的技术。

相当于把一部分原来存在磁盘上的内容加载到了内存中，以此提高访问速率。

物理地址：主存储器的地址。

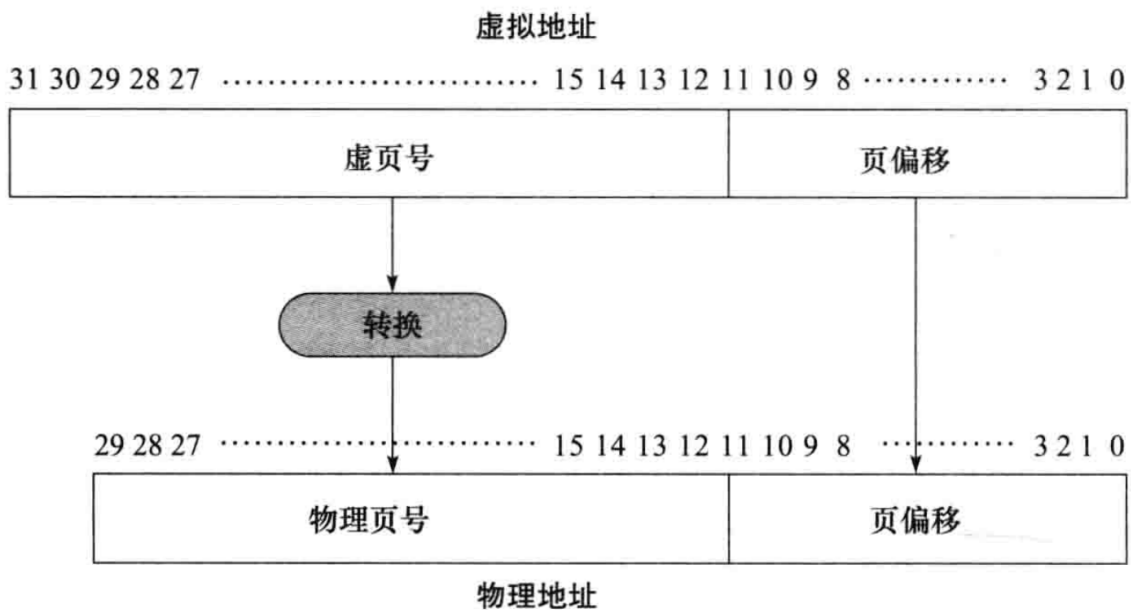
虚拟存储器提供重定位功能，简化指向程序的加载过程，在用地址访存前重定位将出现所用的虚拟地址映射到不同的物理地址，允许程序加载到主存的任意位置。



磁盘和内存之间交换的块称为页，虚拟页号通过查询页表映射到物理页号。由于内存和磁盘中页大小相同，页内偏移和物理页号拼接形成物理地址。一个虚页也可能不在主存中，因此无法映射到物理地址，在这种情况下，页就被存在磁盘上。物理页可以被两个指向相同物理地址的虚拟地址共享，这种方式用来是两个不同的程序共享数据或代码。

虚存中一次缺页处理的代价是百万个时钟周期。

在下图中，页的大小为 $4KiB = 2^{12}B$ ，因此页偏移量有12位。

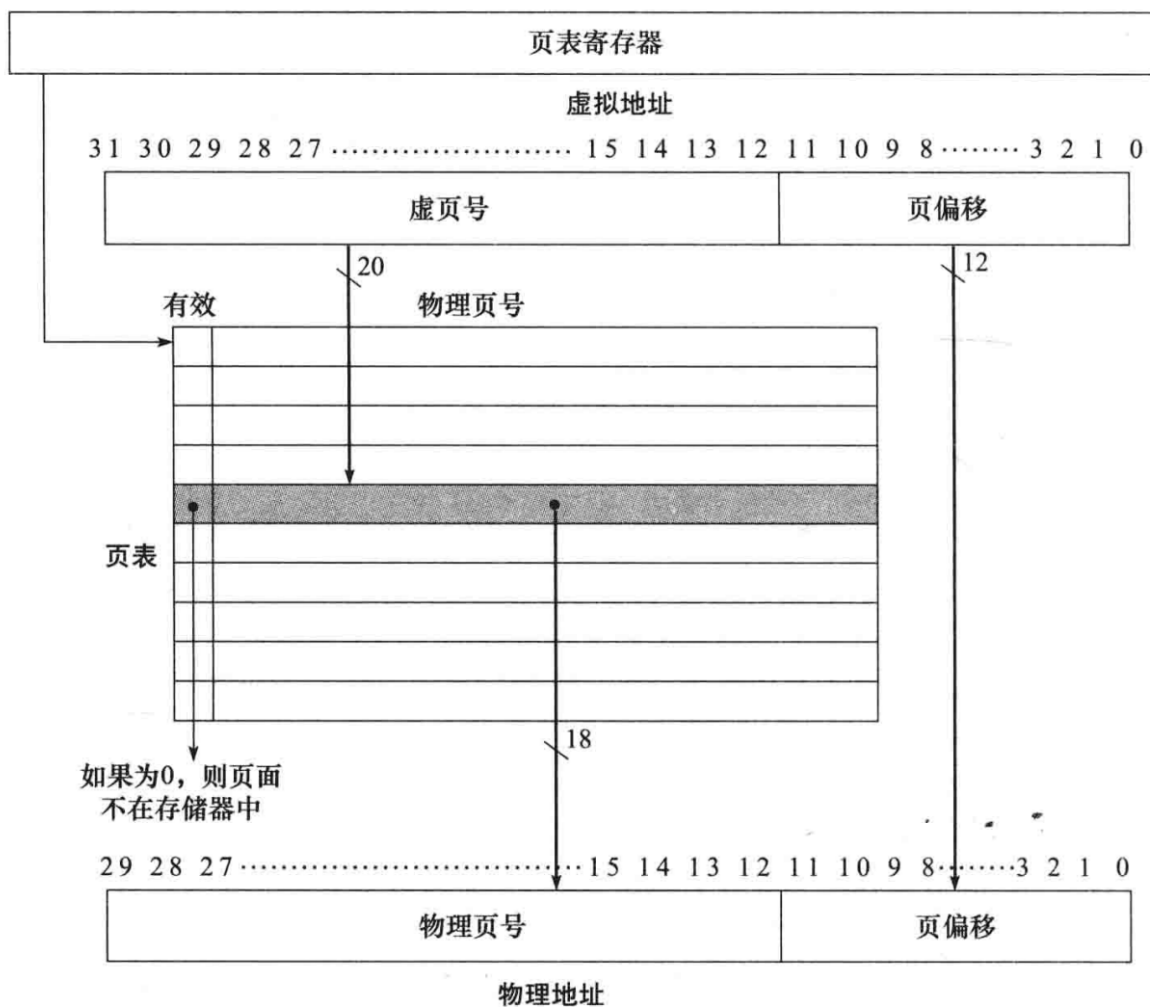


在虚拟存储器中，我们使用各异索引存储器的表来定位页，这种结构被称为页表。页表采取全相联结构。页表使用虚拟地址中的页号作为索引，以找到相应的物理页号。每个进程都有一张页表，存放在自己的内存空间。页表首地址存放在页表寄存器中。

每个页表项使用使用1位有效位，如果该位为无效，该页就不在主存中，就发生一次缺页。如果该页为有效，表明该页在主存中，并且该项包含有物理页号。

缺页通常使用LRU替换策略。

可以采取多级页表或者段页式存储优化存储结构。



TLB

由于页表存放在主存中，因此程序每次访存至少需要两次：第一次访存先获得物理地址，第二次访存才能获得数据。提高访问性能关键在于依靠页表的访问局部性，当一个转换的虚拟页号被使用时，他可能在不久的将来再次被用到，因为对该页中字的引用同时具有时间局部性和空间局部性。

因此，现代处理器都包含一个特殊的cache以跟踪最近使用过的地址变换，这个特殊的地址转换cache通常称为快表(TLB)。TLB在CPU里。

快表：用于记录最近使用地址的映射信息的高速缓存，从而避免每次都要访问页表。

TLB需要将虚拟页号作为标记位，并保留页表中的全部信息。

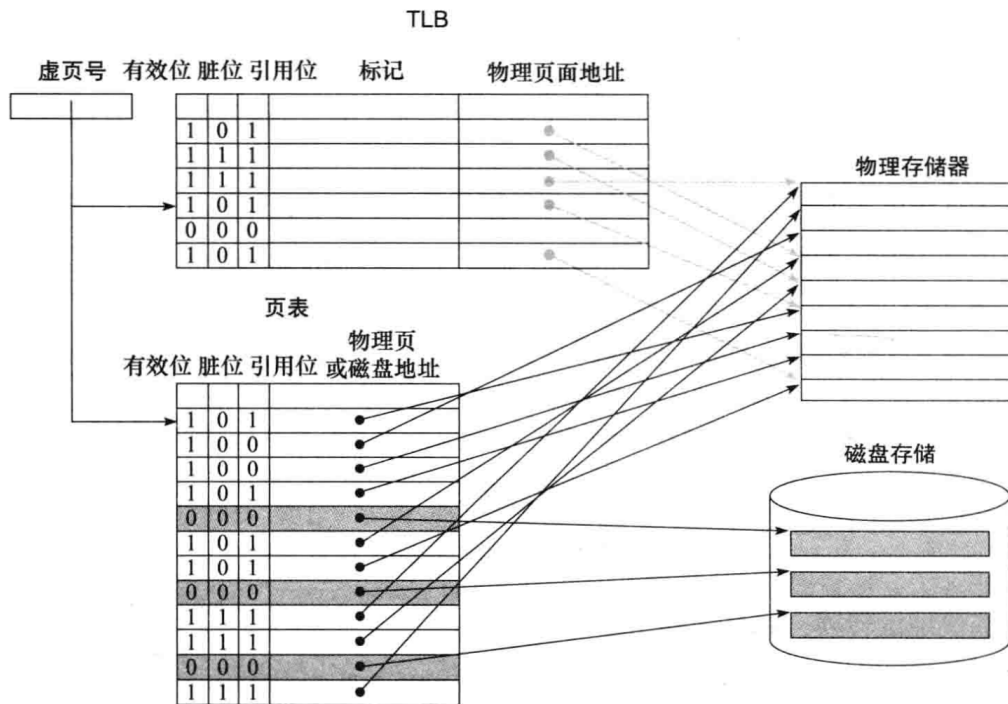


图 5-29 TLB 作为页表的 cache，用于存放映射到物理页中的那些项。TLB 包含了页表中虚页到物理页映射的一个子集。TLB 映射以灰线显示。因为 TLB 是一个 cache，它必须有标记域。如果一个页在 TLB 中没有匹配的项，就必须检查页表。页表或者提供该页的物理页号（可用来创建一个 TLB 项），或者指出该页在磁盘上，这时就会发生缺页。由于页表对于每个虚页都有一个相应的项，并不需要标记；换句话说，不同于 TLB，页表并不是 cache

TLB的缺失有下面两种可能性之一：

1. 页在主存中，只需要创建缺失的TLB表项。
2. 页不在主存中，需要将控制权交给操作系统来解决缺页。

MIPS通常采用软件来处理TLB缺失，他从主存中取出页表项装入TLB，然后重新执行引起TLB缺失的那条指令，这时就会得到TLB命中。如果页表项指出该页不在主存中。此时就会发生缺页异常。

处理TLB缺失或者缺页需要使用异常机制来中断活跃的进程，将控制权传给操作系统，然后恢复执行被中断的进程。一旦操作系统知道了引起缺页的虚拟地址，它必须完成以下三个步骤：

1. 使用虚拟地址查找页表项，并在磁盘上找到被访问页的位置。
2. 选择替换一个物理页；如果被选中的页被修改过，需要在把新的虚拟页装入之前将这个物理页写回到磁盘上。
3. 启动读操作，将被访问的页从磁盘上取回到所选择的物理页的位置上。

当然，最后一个步骤将花费数百万个时钟周期（如果被替换的页被重写过，那么第二步也需要花费这么多时间）。

系统外部辅存（关键问题18）

磁盘RAID

RAID（独立冗余磁盘阵列）是指将多个独立的物理磁盘组成一个独立的逻辑盘，数据在多个物理盘上分割交叉存储、并行访问，具有更好的存储性能、可靠性和安全性。

- RAID0：连续以位或字节为单位分割数据，并行读/写于多个磁盘上，因此具有很高的数据传输率，但它没有数据冗余，因此并不能算是真正的RAID结构。RAID 0只是单纯地提高性能，并没有为数据的可靠性提供保证，而且其中的一个磁盘失效将影响到所有数据。因此，RAID 0不能应用于数据安全性要求高的场合。
- RAID1：它是通过磁盘数据镜像实现数据冗余，在成对的独立磁盘上产生互为备份的数据。当原始数据繁忙时，可直接从镜像拷贝中读取数据，因此RAID 1可以提高读取性能。RAID 1是磁盘阵列中单位成本最高的，但提供了很高的数据安全性和可用性。当一个磁盘失效时，系统可以自动切换到镜像磁盘上读写，而不需要重组失效的数据。
- RAID5：RAID 5不单独指定的奇偶盘，而是在所有磁盘上交叉地存取数据及奇偶校验信息。在RAID 5上，读/写指针可同时对阵列设备进行操作，提供了更高的数据流量。RAID 5更适合于小数据块和随机读写的数据。
- RAID1+0：在连续地以位或字节为单位分割数据并且并行读/写多个磁盘的同时，为每一块磁盘作磁盘镜像进行冗余。它的优点是同时拥有RAID 0的超凡速度和RAID 1的数据高可靠性，但是CPU占用率同样也更高，而且磁盘的利用率比较低。

磁带库

特点：脱机保存、互换读出、存储容量大、价格低廉、携带方便、能效低

场景：低成本

光盘塔

特点：记录密度高、存储容量大高安全、高可靠

场景：一次性写入，不需要经常改写数据

汉明码与奇偶校验（关键问题19）

两个等长二进制数的汉明距离是两个数对应位置不同的位的数量。

错误检测码：这种编码方式能够检测出数据中有1位错误，但是不能对错误位置进行精确定位，因此不能纠正错误。

汉明使用奇偶校验码进行错误检测

- 奇校验：检查数据中1的个数，保证添加完校验位之后1的个数为奇数。如果原本的数据中1的个数为奇数，为了保证添加完校验位后1的个数为奇数，校验位为“0”。
- 偶校验，检查数据中1的个数，保证添加完校验位之后1的个数为偶数。如果原本的数据中1的个数为奇数，为了保证添加完校验位后1的个数为偶数，校验位为“1”。

这种校验方法实际上可以检测出错误位数为奇数个的情况。但是他无法得到错误的实际是哪一个位置。

为了既能检错又能纠错，我们采用额外的检测位确定单个错误的位置，称为汉明纠错码。下面是计算步骤：

1. 对数据部分从左到右由1开始依次编号，这跟通常采用的从最右边开始由0开始编号的做法相反。
2. 将所有编号为2的整数次幂的位置记为奇偶校验位

3. 其他剩余位置用作数据位

4. 奇偶校验位的位置决定了其对应的数据位

位置		1	2	3	4	5	6	7	8	9	10	11	12
编码之后的数据位		p1	p2	d1	p4	d2	d3	d4	p8	d5	d6	d7	d8
奇偶校验 位覆盖 范围	p1	X		X		X		X		X		X	
	p2		X	X			X	X			X	X	
	p4				X	X	X	X					X
	p8								X	X	X	X	X

- 校验位1(0001₂)检查第1, 3, 5, 7, 9, ...位, 这些数位编号的二进制形式最右边一位均为1(0001₂, 0011₂, 0101₂, ...)
 - 校验位2(0010₂)检查第2, 3, 6, 7, 10, 11, 14, 15, ...位, 这些数位编号的二进制形式右边起第二位均为1
 - 校验位4(0010₂)检查第4 ~ 7, 12 ~ 15, 20 ~ 23, ...位, 这些数位编号的二进制形式右边起第三位均为1
 - 校验位8(0010₂)检查第8 ~ 15, 24 ~ 31, 40 ~ 47, ...位, 这些数位编号的二进制形式右边起第四位均为1
5. 设置奇偶校验位, 对每组进行偶校验。如果组成的二进制数(p_8, p_4, p_2, p_1)是0000₂, 这就说明没有发生错误。但是如果校验位组成的二进制数为1010₂, 也就是十进制数10时, 汉明纠错码告诉我们第10位(d6)出错了。

例: 假定存在某个单字节数据10011010₂。首先写出对应的汉明纠错码, 然后把第十位取反, 说明纠错码如何找到并纠正这个错误。

将1, 2, 4, 8位空出来: _ _ 1 _ 001 _ 1010, 分别计算每个校验位

- 校验位1: 现在的数有1, 0, 1, 1, 1, 采用偶校验, 应填入0
- 校验位2: 现在的数有1, 0, 1, 0, 1, 采用偶校验, 应填入1
- 校验位4: 现在的数有0, 0, 1, 0, 采用偶校验, 应填入1
- 校验位8: 现在的数有1, 0, 1, 0, 采用偶校验, 应填入0

最终得到的码数为011100101010, 把第十位反转之后得到的数为011100101110。

- 校验位1为0 (011100101110, 4个1, 偶性)
- 校验位2为1 (011100101110, 5个1, 奇性, 有错)
- 校验位3为0 (011100101110, 2个1, 偶性)
- 校验位4为1 (011100101110, 3个1, 奇性, 有错)

校验码2和10位不正确, 因为2+8=10, 所以第十位是错的, 将其反转为0, 得到011100101010

汉明码没有止步于1位纠错码。通过付出增加一位的代价, 可以让码组中的最小汉明距离变到4, 这就意味着可以做到纠正1位错误并检测2位错误。该方法在数据的末尾增加了1位奇偶校验码, 对整个数据进行计算校验。

汉明并没有止步于1位纠错码。通过付出增加一位的代价，可以让码组中的最小汉明距离变到4。这就意味着我们可以做到纠正1位错并检测2位错。该方法增加了1位奇偶校验码，对整个字进行计算校验。这里我们以4位的数据字为例，这只需要7位就能完成单位错检测。计算出汉明奇偶校验位 $H(p_1 p_2 p_3)$ ，这里仍然采用偶校验，最后计算出整个字的偶校验位 p_4 ：

1	2	3	4	5	6	7	8
p_1	p_2	d_1	p_3	d_2	d_3	d_4	p_4

上述用于纠正1位错同时检测2位错的算法仍像之前那样先计算出纠错码组的奇偶性 H ，最后再计算一下全组的奇偶校验位 p_4 就可以了。以下是可能出现的4种情况：

- 1) H 为偶并且 p_4 为偶，这表示没有错误发生。
- 2) H 为奇并且 p_4 为奇，这表明出现了一位可纠正错误。（当出现一位错时， p_4 应当为奇。）
- 3) H 为偶并且 p_4 为奇，这说明出现的仅仅是 p_4 ，因此将 p_4 取反即可。
- 4) H 为奇并且 p_4 为偶，这表示出现了两位错。（当出现两位错时， p_4 应当为偶。）

六、其他

谬误与陷阱（关键问题20）

第一章

陷阱：在改进计算机的某个方面时期望总性能的提高与改进大小成正比。

拿前面的例子看，假设一个程序在一台计算机上运行需要100秒，其中80秒用于乘法操作，如果想把该程序的运行速度提高到5倍，那么无论怎么改进乘法操作都做不到。

Amdahl定律阐述了“对于特定改进的性能提升可能由所使用的改进特征的数量所限制”的规则。它是“收益递减定律”的量化版本。

谬误：利用率低的计算机功耗低。

低利用率，功耗不一定低。

谬误：面向性能的设计和面向能量效率的设计具有不相关的目标。

由于能耗时功耗和时间的乘积，在通常情况下，对于软硬件的优化而言，即使在优化的部分起作用时能耗可能高了一些，但是这些优化缩短了系统运行时间，因此整体上还是节约了能量。一个重要的原因是当一个程序运行时，计算机的其他部分仍在消耗能量，因此，即使优化的部分多消耗了能量，运行时间的减少也可以减少整个系统的能耗。

陷阱：用性能公式的一个子集去度量性能。

比如前面的使用MIPS度量计算机性能存在三个问题：

- MIPS规定了指令执行的速率，没有考虑指令的能力，无法比较不同指令集的计算机。
- 在同一台计算机上，不同程序会有不同的MIPS，因此一台计算机会有不同的MIPS值。
- 如果一个程序执行的指令数更多，但每条指令的执行速度更快，则MIPS的变化是与性能无关的。

第二章

谬误：更强大的指令意味着更高的性能。

x86可以通过前缀来改变后续指令的执行，某个前缀可以重复执行后面的指令直到一个计数器减少至0，这个指令可以用来实现32位的存储器到存储器的传输。但是也可以通过代码复制的形式减少循环开销，这种方式比前者快了1.5倍。也可以使用更大的浮点寄存器代替x86的整数寄存器，比使用复杂指令快了2倍。

谬误：使用汇编语言编程来获得最高的性能。

编译器产生的汇编代码与手工编写的代码之间的差距在快速缩小。高级语言可以使用编译器使得同样的代码可以在不同机器上运行，还可以使软件便于维护。

谬误：商用二进制兼容的重要性意味着成功的指令集不需要改变。

x86指令集也在逐渐增加新的指令。

陷阱：忘记在字节寻址的机器中，连续的字地址相差不是1。

下一个字地址需要将寄存器的值加4。

陷阱：在自动变量定义过程外，使用指针指向该变量。

处理指针的常见错误是使用指向一个过程中局部数组的指针，从该过程传出结果。由于局部数组在栈中，当过程返回时，包含局部数组的存储器会被重新使用，导致指针发生混乱。

第三章

谬误：右移指令可以代替与2的幂相除。

对于无符号整数确实如此，但是对于有符号整数却有问题。比如 -5_{10} ，右移两位后最高两位变成0，显然不对。也可以进行算术右移，使用符号位扩展填充，但是这样得到的结果是 -2_{10} 而不是 -1_{10} ，依然存在问题。

陷阱：浮点数加法是不能使用结合律的。

由于计算机计算精度优先，结合律不能适用于浮点数，当两个不同符号大数与一个小数相加时就会出现問題。

谬误：并行执行策略不但适用于整型数据类型，同样也适用于浮点数据类型。

由于浮点数加法不能结合，所以浮点数并行计算得到的结果可能不同。

陷阱：MIPS指令 `addiu`（无符号立即数加）会对16位立即数域进行符号扩展。

当我们不关心上溢时，`addiu`经常用于将常数和有符号整数相加。由于MIPS没有立即数减的指令，所以MIPS体系结构设计者决定对该指令的立即数域进行符号扩展，以支持立即数为负数时的需要。

第四章

谬误：流水线是一种简单的结构。

流水线其实十分复杂，设计流水线必须非常小心。

谬误：流水线概念的实现与工艺无关。

当芯片上晶体管的数量和速度决定五级流水线是最好的解决方案时，延迟分支是一种简单的控制冒险的方法。但对于长流水线、超标量执行和动态分支预测，延迟分支就成为多余的方法了。

陷阱：没有考虑指令集的设计反过来会影响流水线。

许多流水线中遇到的困难都是由指令集的复杂性造成的。

第五章

陷阱：在写程序或编译器生成代码时忽略存储系统的行为。

如果程序员在设计算法时考虑存储系统的行为，则可很容易地将性能翻倍。

陷阱：在模拟cache的时候，忘记说明字节编址或者cache块大小。

当模拟cache（手动或者通过计算机），我们必须保证，在确定一个给定的地址被映射到哪个cache块中时，一定要说明字节编址和多字块的影响。例如，如果我们有一个容量为32字节的直接映射的cache，块大小为4字节，则字节地址36映射到cache的块1，因为字节地址是36，转为为块地址时9，而 $9 \bmod 8 = 1$ 。另一方面，如果地址36是字地址，那么他就映射到块 $36 \bmod 8 = 4$ 。要保证清楚地说明基地址。

陷阱：对于共享cache，组相联度少于核的数量或者共享该cache的线程数。

如果不特别注意，一个运行在 2^n 个处理器或者线程上的并程序为数据结构分配的地址可能映射到共享二级cache同一个组中。如果cache至少是 2^n 路组相联，那么通过硬件可以隐藏这些程序偶尔发生的冲突，如果不是，那么就会发生性能缺陷。

陷阱：用存储器平均访问时间来评估乱序处理器的存储器层次结构。

如果处理器在cache缺失时阻塞，那么你可以分别计算存储器阻塞时间和处理器执行时间，因此可以使用存储器平均访问时间来独立地评估存储器层次结构。

如果处理器在cache缺失时继续执行指令，而且甚至可能维持更多的cache缺失，那么唯一可以用来准确评估存储器层次结构的办法是模拟乱序处理器和存储器结构。

陷阱：通过在未分段地址空间的顶部增加段来扩展地址空间。

这样在大数组的索引、无限制的指针或者是引用参数的时候出问题。增加段可以将每个地址变成两个字——一个是段号，另一个是段内偏移——这些在使用寄存器中的地址时就会出现这个问题。

谬误：实际的磁盘故障率和规格书中声明的一致。

实际的磁盘故障率会比声明的多。

谬误：操作系统是调度磁盘访问最好的地方。

由于磁盘知道逻辑地址被映射到的扇区、磁道及磁面上的实际的物理地址，这样通过调度可以减少旋转以及寻道的时间。但是操作系统可能会对一些操作进行重排序，这样重排序得到的结果可能更糟。

陷阱：在不为虚拟化设计的指令集体系结构上实现虚拟机监视器。

有些指令在这种情况下会出现问题。

书后习题

2.27

题目：将下面的C代码翻译成MIPS汇编代码。要求使用的指令数目最少。假设值a、b、i和j分别存放在寄存器\$s0、\$s1、\$t0和\$t1中。另外假设寄存器\$s2中存放着数组D的基地址。

```
1  for (i = 0; i < a; i++)
2      for (j = 0; j < b; j++)
3          D[4 * j] = i + j;
```

答案：

```
1  addi $t0,$zero,0 #int i=0
2  LOOP1:
3  slt $t2,$t0,$s0 #判断i是否小于a
4  beq $t2,$zero,DONE1 #若i<a为假，跳出循环
5  addi $t1,$zero,0 # int j = 0
6  LOOP2:
7  slt $t3,$t1,$s1 #判断j是否小于b
8  beq $t3,$zero,DONE2 #若j<b为假，跳出循环
9  add $t4,$t0,$t1 #计算i+j
10  sll $t5,$t1,2 #计算4*j，存入$t5
11  add $t5,$t5,$s2 #计算D[4*j]地址
12  sw $t4,$t5 #将i+j存入D[4*j]
13  addi $t1,$t1,1 #j++
14  j LOOP2
15  DONE2:
16  addi $t0,$t0,1 #i++
17  j LOOP1
18  DONE1:
```

3.43

题目：写出数值1/3的尾数的位模式，其浮点格式采用二进制编码的尾数。假设有24位，并且不需要进行规格化。这种表达精确吗？

答案：尾数为0101 0101 0101 0101 0101 0101。这种表达不精确，因为1/3是个无限小数，这种表达转换成小数是个有限小数。

4.8

题目：本练习题讨论流水线对处理器时钟周期的影响。下面给出了数据通路中不同阶段的延迟情况：

IF	ID	EX	MEM	WB
250ps	350ps	150ps	300ps	200ps

另外，假定处理器执行各种指令的比率如下面所示：

alu	beq	lw	sw
45%	20%	20%	15%

- 4.8.1 [5] <4.5> 流水线处理器与非流水线处理器的时钟周期分别是多少？
- 4.8.2 [10] <4.5> lw指令在流水线处理器和非流水线处理器中的总延迟分别是多少？
- 4.8.3 [10] <4.5> 如果可以将原流水线数据通路的一级划分为两级，每级的延迟是原级的一半，那么你会选择哪一级进行划分？划分后处理器的时钟周期为多少？
- 4.8.4 [10] <4.5> 假设没有阻塞和冒险，数据存储器的利用率是多少（占总周期数的百分比）？
- 4.8.5 [10] <4.5> 假设没有阻塞和冒险，寄存器堆的写寄存器端口的利用率是多少？
- 4.8.6 [30] <4.5> 假设一种多周期的处理器设计，其中每条指令需要多个时钟周期完成，但上一条指令完成前不取下一条指令。在这种设计中，指令仅经过其所需的阶段（例如，存储指令仅需4个时钟周期，因为其不需要WB阶段）。比较单周期设计、多周期设计和流水线设计三者的时钟周期和总执行时间。

答案：

- 流水线的时钟周期等于耗时最长的阶段，即ID阶段，所以时钟周期为350ps
非流水线的时钟周期等于各阶段时间之和，也就是 $250 + 350 + 150 + 300 + 200 = 1250ps$
- 在流水线处理器的总延迟等于 $5 \times 350 = 1750ps$ ，在非流水线中总延迟为1250ps
- 由于ID级耗时最长，所以将其划分为两极，在划分之后的时间最长的阶段是MEM的300ps，所以时钟周期为300ps
- 使用到数据存储器的指令为lw指令（读数据存储器）和sw指令（写数据存储器），所以利用率为 $20\% + 15\% = 35\%$
- 由于alu会将计算结果写回寄存器，lw会将数据寄存器里取出的数据写入寄存器中，所以利用率为 $45\% + 20\% = 65\%$
- 单周期时钟周期为1250ps，总执行时间为1250n ps(n为指令数)
流水线时钟周期为350 ps，总执行时间为350n ps(n为指令数)
多周期时钟周期为350ps，其中alu、beq和sw指令只需要四个时钟周期，lw指令需要5个时钟周期，所以总执行时间为 $(0.8 \times 4 + 0.2 \times 5) \times 350n = 1470n ps$

4.14

题目：本练习题讨论流水线处理器的延迟时间槽、控制冒险和分支执行之间的关系。假设下列MIPS代码在一个五级流水线、有完全旁路和预测分支总发生的处理器上运行。试根据下面的两个序列回答下列问题。

```

1      lw r2,0(r1)
2  label1: beq r2,r0,label2 #not taken once,then taken
3      lw r3,0(r2)
4      beq r3,r0,label1 #taken
5      add r1,r3,r1
6  label2: sw r1,0(r2)

```

- 4.14.1 [10] <4.8> 画出该指令序列的流水线执行图，假设没有延迟时间槽而且在 EX 级执行分支。
- 4.14.2 [10] <4.8> 重做练习题 4.14.1，但是假设使用了延迟时间槽。给定的代码中，跟在分支之后的指令是该分支的延迟槽指令。
- 4.14.3 [20] <4.8> 不同于需要一个 ALU 操作，另一种提前确定分支方向的方法是使用 “bez rd,label” 和 “bnez rd,label” 之类的条件分支指令，该类条件分支指令根据寄存器值是否为零决定是否分支。变换原指令序列以使用该类条件分支指令而非 beq 指令。假设寄存器 \$8[⊖] 是临时寄存器，并且可以使用 R 型指令 seq (set if equal, 相等置位)。
- 4.8 节说明了如何把分支执行提前到 ID 级以减少控制冒险。这个方法需要在 ID 级增加一个专用的比较器，如图 4-62 所示。但是这个方法增加了 ID 级的延迟，并且需要额外的旁路逻辑和冒险检测。
- 4.14.4 [10] <4.8> 以指令序列中第一条分支指令为例，说明图 4-62 中为支持在 ID 级执行分支应加入的冒险检测逻辑。该逻辑需要检测什么类型的冒险？
- 4.14.5 [10] <4.8> 对给定的指令序列，把执行分支移到 ID 级带来的加速比是多少？为什么？假设 ID 级进行的额外比较不影响时钟周期。
- 4.14.6 [10] <4.8> 以指令序列中第一条分支指令为例，说明为支持在 ID 级执行分支应加入的旁路。比较新旁路单元与图 4-62 中旁路单元的复杂度。

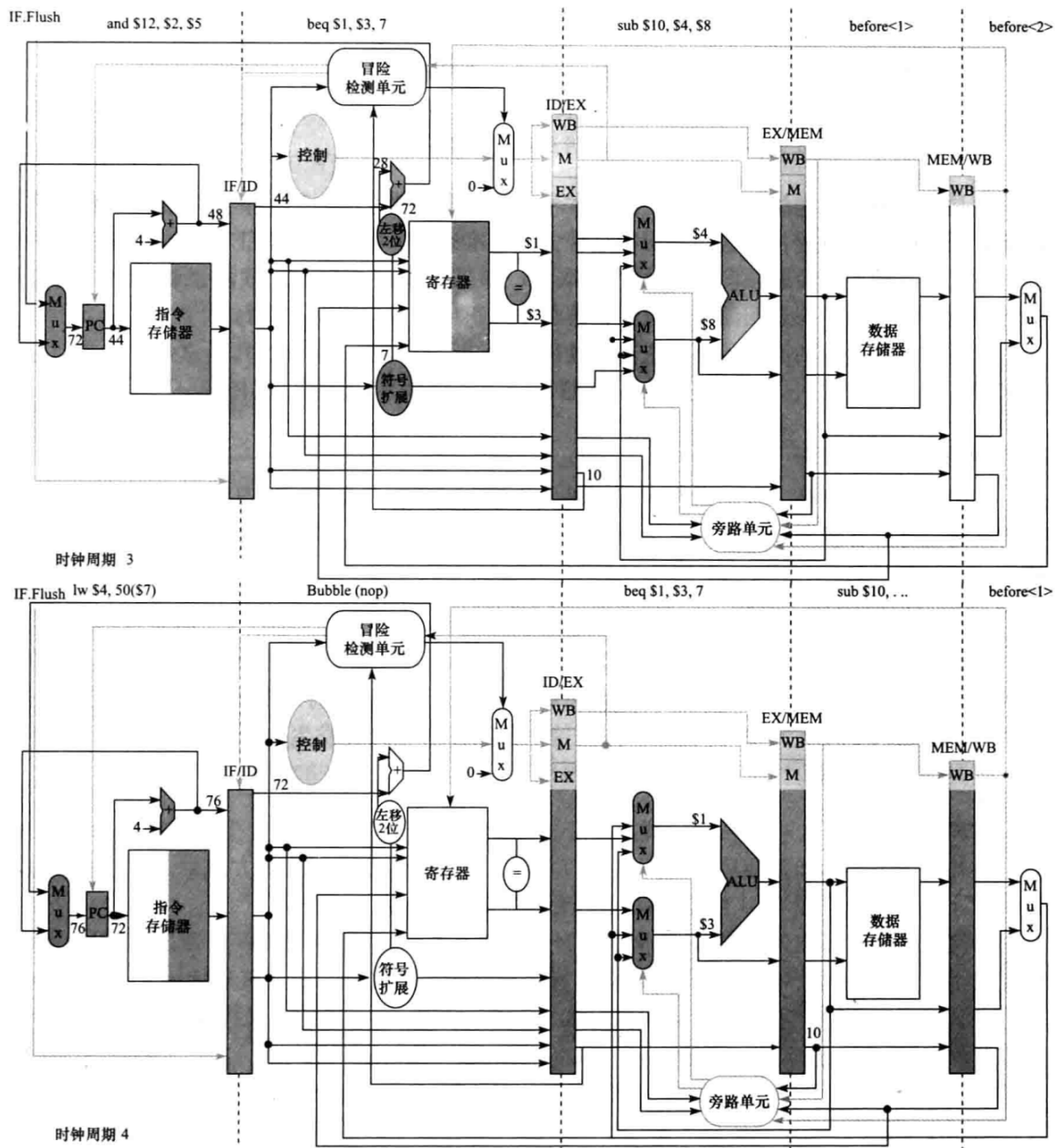


图 4-62 在第三个时钟周期 ID 级确定分支发生，因此地址 72 被选为下一个 PC 地址，同时将为下一个时钟周期预取的指令置为 0。时钟周期 4 的图描述了地址为 72 的指令被取回，并且分支发生的后果是在流水线中产生了一个气泡或者一条空指令（由于空指令实际上是 `sll $0, $0, 0`，所以时钟周期 4 的 ID 级是否应该标出还有待商榷）

答案：

1.	1	2	3	4	5	6	7	8	9	10	11	12	13	14
1 lw r2,0(r1)	IF	ID	EX	MEM	WB									
2 beq r2,r0,label2		IF	ID	-	EX	MEM	WB							
3 lw r3,0(r2)						IF	ID	EX	MEM	WB				
4 beq r3,r0,label1							IF							
5 beq r2,r0,label2								IF	-	ID	EX	MEM	WB	
6 sw r1,0(r2)										IF	ID	EX	MEM	WB

分析：

- 在指令1和指令2之间，由于题目要求beq在EX级执行分支，所以beq的EX级需要在lw的MEM级之后执行（取数-使用数据冒险），所以需要阻塞一个时钟周期。
- 在指令2和指令3之间，由于该处的beq没有跳转，与预测的策略相反，所以指令3需要在指令2的EX段之后执行。
- 在指令3和指令4之间，同样出现了取数-使用数据冒险，所以需要阻塞一个时钟周期。
- 在指令4和指令5之间，由于这里的分支被采取，与预测策略相同，所以不需要阻塞，直接进入流水线。后面同理。

2.		1	2	3	4	5	6	7	8	9	10	11	12	13	14
1	lw r2,0(r1)	IF	ID	EX	MEM	WB									
2	beq r2,r0,label2		IF	ID	-	EX	MEM	WB							
3	lw r3,0(r2)			IF	-	ID	EX	MEM	WB						
4	beq r3,r0,label1						IF	ID	EX	MEM	WB				
5	add r1,r3,r1							IF	ID	EX	MEM	WB			
6	beq r2,r0,label2								IF	ID	EX	MEM	WB		
7	lw r3,0(r2)									IF	ID	EX	MEM	WB	
8	sw r1,0(r2)										IF	ID	EX	MEM	WB

分析：和上一问相比，这里多了指令5和指令7，因为在分支指令后的一条指令会被放入延迟时间槽。

- 这里1、2指令阻塞，因为有取数-使用数据冒险。
- 这里3、4发生阻塞，因为要在2的EX之后执行。
- 后续因为有了延迟时间槽的加入，所以没有阻塞的情况发生。

```

3. 1  lw r2,0(r1)
    2  label1: seq $8,r2,r0
    3      benz $8,label2 #not taken once,then taken
    4      lw r3,0(r2)
    5      seq $8,r3,r0
    6      benz $8,r0,label1 #taken
    7      add r1,r3,r1
    8  label2: sw r1,0(r2)

```

4. 冒险检测逻辑必须检测到以下情况：分支指令依赖于先前的R型指令的结果，或者依赖于前两个加载指令的结果。

- 当分支指令在其ID阶段使用其寄存器操作数的值时，R型指令的结果仍在EX段。因此，我们必须暂停处理器，并在下一个周期重复分支指令的ID段。
- 如果分支指令依赖于紧随其后的加载指令，加载指令的结果在分支进入ID段后仅生成两个周期，因此我们必须暂停分支指令两个周期。
- 如果分支指令依赖于倒数第二个加载指令，当分支指令处于其ID段时，加载指令正在MEM段，因此我们必须暂停分支指令一个周期。

注意在这三种情况下，我们假设前面指令的值在可能的情况下被转发到分支指令的ID阶段。

5.		1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
1	lw r2,0(r1)	IF	ID	EX	MEM	WB										
2	beq r2,r0,label2		IF	-	-	ID	EX	MEM	WB							
3	lw r3,0(r2)						IF	ID	EX	MEM	WB					
4	beq r3,r0,label1							IF	-	-	ID	EX	MEM	WB		
5	beq r2,r0,label2										IF	ID	EX	MEM	WB	
6	sw r1,0(r2)											IF	ID	EX	MEM	WB

加速比是14/15=0.93。图中流水线要经过15个时钟周期，但是原本只需要14个时钟周期。

6. 分支指令现在在ID阶段执行。如果分支指令在ID阶段使用了由紧随其后的指令产生的寄存器值，就像我们在题4部分中描述的那样，由于在分支使用过时的寄存器值时前一条指令正在EX阶段执行，因此必须暂停分支指令。如果ID阶段的分支指令依赖于MEM阶段的R型指令，我们需要进行转发以确保分支指令的正确执行。同样地，如果ID阶段的分支指令依赖于WB阶段的R型或加载指令，我们需要进行转发以确保分支指令的正确执行。总体而言，我们需要另一个转发单元，其输入与转发到EX阶段的单元相同。新的转发单元应该控制两个放置在分支比较器之前的多路选择器(Mux)。每个多路选择器在寄存器中选择读取的值，EX/MEM流水线寄存器中的ALU输出以及MEM/WB流水线寄存器中的数据值之间进行选择。新的转发单元的复杂性与现有单元的复杂性相同。

4.18

题目：本练习题比较单发射和双发射处理器的性能，并考虑对双发射处理器进行程序优化。根据下表的C代码分别回答下列问题。

```
1  for (i = 0; i != j; i += 2)
2      b[i] = a[i] - a[i + 1];
```

在编写MIPS代码时，假设变量被保存在寄存器中，如下表所示，除了空闲的寄存器，其余寄存器都用来保存变量的值，因此不能再用作其他用途。

i	j	a	b	c	空闲
R5	R6	R1	R2	R3	R10, R11, R12

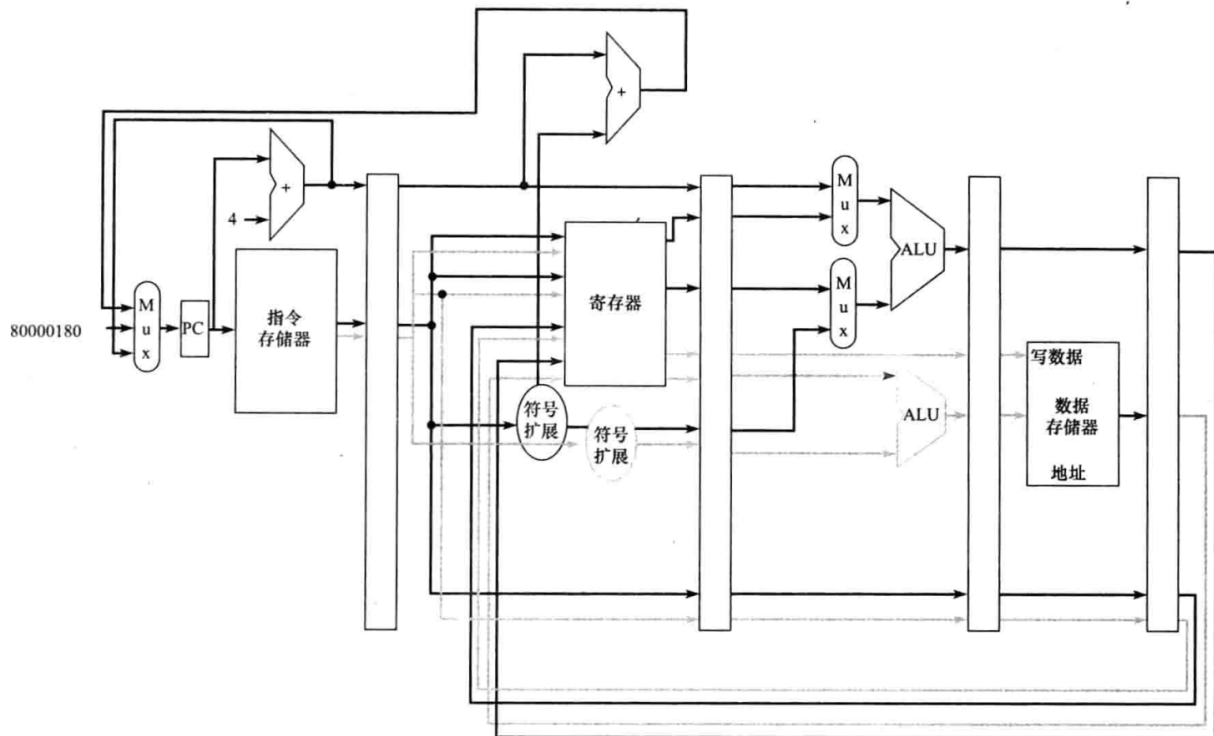


图 4-69 一个静态双发射的数据通路。双发射所需的额外硬件用灰色线显示，主要包括：来自指令存储器的额外 32 位输出，寄存器堆多出的两个读端口和一个写端口，还有一个额外的 ALU。这里假设下面那个 ALU 处理数据传输时的地址计算，而上面那个 ALU 处理所有的其他操作

1. 将这段代码翻译成MIPS代码。这种翻译必须是直接的，不允许对代码进行重排序以达到更好的性能。

- 如果循环仅执行两次后就退出，画出题1中MIPS代码在图4-69的双发射处理器中执行的流水线图。假设处理器能进行完美的分支预测，并且一个周期能取任意两条指令（不仅仅是连续的两条指令）。
- 重排序题1中的MIPS代码，以在图4-69的双发射处理器静态调度处理器上获得更好的性能。
- 重做题2，但这次使用题3中的MIPS代码。
- 从单发射处理器到图4-69的双发射处理器，性能的加速比是多少？在单发射和双发射处理器分别运行题1中的代码，假设循环执行1 000 000次。与题2相同，假设处理器能进行完美的分支预测，并且一个双发射处理器在同一个周期能取任意两条指令。
- 重做题5，这次假设双发射处理器中的一条指令可以是任意类型的，而另一条指令必须是非存取指令。

答案：

```

1.  1  addi R5,$zero,0 # i=0
    2  Again:
    3  beq R5,R6,End # i != j
    4  add R10,R5,R1 # a[i]
    5  lw R11,0(R10) # 取a[i]
    6  lw R10,1(R10) # 取a[i+1]
    7  sub R10,R11,R10 # a[i] - a[i+1]
    8  add R11,R2,R5 # b[i]
    9  sw R10,0(R11) # 存入b[i]
   10  addi R5,R5,2 # i+=2
   11  j Again # 循环
   12  End:

```

- 将图中的 BEW R0,R0,Again 换成 j Again

Instructions	Pipeline
ADD R5,R0,R0	IF ID EX ME WB
BEQ R5,R6,End	IF ID ** EX ME WB
ADD R10,R5,R1	IF ** ID EX ME WB
LW R11,0(R10)	IF ** ID ** EX ME WB
LW R10,1(R10)	IF ** ID EX ME WB
SUB R10,R11,R10	IF ** ID ** ** EX ME WB
ADD R11,R5,R2	IF ** ** ID EX ME WB
SW R10,0(R11)	IF ** ** ID ** EX ME WB
ADDI R5,R5,2	IF ** ID EX ME WB
BEW R0,R0,Again	IF ** ID ** EX ME WB
BEQ R5,R6,End	IF ** ID EX ME WB
ADD R10,R5,R1	IF ** ID ** EX ME WB
LW R11,0(R10)	IF ** ID EX ME WB
LW R10,1(R10)	IF ** ID ** EX ME WB
SUB R10,R11,R10	IF ** ID ** EX ME WB
ADD R11,R5,R2	IF ** ID ** ** EX ME WB
SW R10,0(R11)	IF ** ** ID EX ME WB
ADDI R5,R5,2	IF ** ** ID EX ME WB
BEW R0,R0,Again	IF ID EX ME WB
BEQ R5,R6,End	IF ID ** EX ME WB

```

3. 1  addi R5,$zero,0 # i=0
    2  Again:
    3  add R10,R5,R1 # a[i]
    4  beq R5,R6,End # i != j
    5  lw R11,0(R10) # 取a[i]
    6  add R12,R2,R5 # b[i]
    7  lw R10,1(R10) # 取a[i+1]
    8  addi R5,R5,2 # i+=2
    9  sub R10,R11,R10 # a[i] - a[i+1]
   10  sw R10,0(R11) # 存入b[i]
   11  j Again # 循环
   12  End:

```

4. 将图中的 BEQ R0,R0,Again 换成 j Again

Instructions	Pipeline
ADD R5,R0,R0	IF ID EX ME WB
ADD R10,R5,R1	IF ID ** EX ME WB
BEQ R5,R6,End	IF ** ID EX ME WB
LW R11,0(R10)	IF ** ID EX ME WB
ADD R12,R5,R2	IF ID EX ME WB
LW R10,1(R10)	IF ID EX ME WB
ADDI R5,R5,2	IF ID EX ME WB
SUB R10,R11,R10	IF ID ** EX ME WB
SW R10,0(R12)	IF ** ID EX ME WB
BEQ R0,R0,Again	IF ** ID EX ME WB
ADD R10,R5,R1	IF ** ID EX ME WB
BEQ R5,R6,End	IF ** ID ** EX ME WB
LW R11,0(R10)	IF ** ID EX ME WB
ADD R12,R5,R2	IF ** ID EX ME WB
LW R10,1(R10)	IF ID EX ME WB
ADDI R5,R5,2	IF ID EX ME WB
SUB R10,R11,R10	IF ID ** EX ME WB
SW R10,0(R12)	IF ID ** EX ME WB
BEQ R0,R0,Again	IF ** ID EX ME WB
ADD R10,R5,R1	IF ** ID ** EX ME WB
BEQ R5,R6,End	IF ** ID EX ME WB

5. 单发射的处理器CPI为1.11，双发射处理器的CPI为1.06，加速比为1.05

6. 单发射的处理器CPI为1.11，双发射处理器的CPI为0.83，加速比为1.34

5.2

5.2 cache 为处理器提供了一个高性能的存储器层次结构，因此十分重要。下面是一个 32 位存储器地址引用的列表，给出的是字地址。

3, 180, 43, 2, 191, 88, 190, 14, 181, 44, 186, 253

5.2.1 [10] <5.3> 已知一个直接映射的 cache，有 16 个块，块大小为 1 个字。对于每次访问，请标识出二进制地址、标记以及索引。假设 cache 最开始为空，那么请列出每次访问是命中还是缺失。

5.2.2 [10] <5.3> 已知一个直接映射的 cache，有 8 个块，块大小为 2 个字。对于每次访问，请标识出二进制地址、标记以及索引。假设 cache 最开始为空，那么请列出每次访问是命中还是缺失。

5.2.3 [20] <5.3, 5.4> 对已知的访问来优化 cache 的设计。这里有三种直接映射的 cache 设计方案，每个容量都为 8 个字：C1 块大小为 1 个字，C2 块大小为 2 个字，C3 块大小为 4 个字。根据缺失率，哪种 cache 设计最好？如果缺失阻塞时间为 25 个周期，C1 的访问时间为 2 个周期，C2 为 3 个周期，C3 为 5 个周期，那么哪种 cache 设计最好？

这里有许多对 cache 整体性能很重要的不同的设计参数。下面列出了对于不同的直接映射 cache 设计的参数。

cache 数据量：32KiB

cache 块大小：2 个字

cache 访问时间：1 个周期

5.2.4 [15] <5.3> 假定 32 位的地址，计算上面列出的 cache 所需的总位数。给定总的大小，找出最接近的直接映像 cache 的总的大小，该 cache 块的大小为 16 个字长或更大。请解释为什么第二种 cache 比第一种 cache 的访问速度更慢，尽管第二种 cache 的数据量更大。

5.2.5 [20] <5.3, 5.4> 在一个 2KiB 的两路组相联 cache 上产生一系列读请求时的缺失率要比在表中 cache 上执行读请求的缺失率低。请给出一个可能的解决方案，使得表中列出的 cache 的缺失率等于或者低于 2KiB cache 的缺失率。讨论这种解决方案的优点和缺点。

5.2.6 [15] <5.3> 5.3 节的公式说明了用来索引直接映射 cache 的典型方法：(块地址) mod (cache 中的块数)。假设地址为 32 位，cache 中有 1024 个块，考虑一个不同的索引函数：(块地址 [31: 27] XOR 块地址 [26: 22])。可以使用这个公式来索引直接映射的 cache 吗？如果可以，请解释原因，并且讨论可能需要对 cache 做的一些改动。如果不可以，请解释原因。

答案：

1.	字地址	二进制地址	索引	标记	H/M
	3	00000011	3	0	M
	180	10110100	4	11	M
	43	00101011	11	2	M
	2	00000010	2	0	M
	191	10111111	15	11	M
	88	01011000	8	5	M
	190	10111110	14	11	M
	14	00001110	14	0	M
	181	10110101	5	11	M
	44	00101100	12	2	M
	186	10111010	10	11	M

字地址	二进制地址	索引	标记	H/M
253	11111101	13	15	M

2.

字地址	二进制地址	索引	标记	H/M
3	00000011	1	0	M
180	10110100	2	11	M
43	00101011	5	2	M
2	00000010	1	0	H
191	10111111	7	11	M
88	01011000	4	5	M
190	10111110	7	11	H
14	00001110	3	0	M
181	10110101	2	11	H
44	00101100	6	2	M
186	10111010	5	11	M
253	11111101	6	15	M

Word Address	Binary Address	Tag	Cache 1		Cache 2		Cache 3	
			index	hit/miss	index	hit/miss	index	hit/miss
3	0000 0011	0	3	M	1	M	0	M
180	1011 0100	22	4	M	2	M	1	M
43	0010 1011	5	3	M	1	M	0	M
2	0000 0010	0	2	M	1	M	0	M
191	1011 1111	23	7	M	3	M	1	M
88	0101 1000	11	0	M	0	M	0	M
190	1011 1110	23	6	M	3	H	1	H
14	0000 1110	1	6	M	3	M	1	M
181	1011 0101	22	5	M	2	H	1	M
44	0010 1100	5	4	M	2	M	1	M
186	1011 1010	23	2	M	1	M	0	M
253	1111 1101	31	5	M	2	M	1	M

3.

Cache1的缺失率为100%

Cache2的缺失率为83%

Cache3的缺失率为92%

上述访问序列访问Cache1的时钟周期为 $25 \times 12 + 2 \times 12 = 324$

访问Cache2的时钟周期为 $25 \times 10 + 3 \times 12 = 286$

访问Cache3的时钟周期为 $25 \times 11 + 5 \times 12 = 335$

4. 一个Cache块的大小为2字，也就是8字节，总数据量为 $32KiB = 32 \times 1024B$ ，所以Cache块的个数为 $32/8 \times 1024 = 4096 = 2^{12}$ 个，所以索引位有12位，32位地址中需要减去2位字节偏移，1位字偏移，12位索引，所以标记位为 $32 - 15 = 17$ 位，还需要1位有效位，所以额外的位数为 $(17 + 1) \times 4096 = 73728Bit = 9216B$ ，所以总的字节数为 $9216 + 32 \times 1024 = 41984B$ ，总位数为335872位。

设Cache的块数为 2^x 个，每个块的大小为16字，那么总的数据量为 $2^x \times 16 \times 4 = 2^{x+6}B$ ，索引有 x 位，字节偏移2位，字偏移4位，所以标记位有 $32 - 6 - x = 26 - x$ 位，还有1位标记位，那么额外的数据量为 $(26 - x + 1) \times 2^x / 8 = (27 - x)2^{x-3}B$ ，所以 $(27 - x)2^{x-3} + 2^{x+6} \geq 41984$ ，解得 $x = 10$ ，这时总的大小为67712B。

原因：块更大，可能需要增加命中时间和未命中时的阻塞时间；同时块更少，缺失率也会更高。

5. 先解释为什么表中的cache命中率会比两路组相联形式低。我们可以构造一个访问序列0, 32768, 0, 32768, ..., 其中0和32768都会映射到索引为0的cache块中。这个序列使用表中的cache每次都会发生miss，但是使用两路组相联的cache只有前两次会发生miss。

解决方案：可以将表中的cache改成两路组相联的cache。优点是提高了命中率，缺点是降低了命中时间。

6. 可以使用该函数进行直接映射。但是，由于异或操作使得有5位的信息发生丢失，所以需要增加更多的标记位在缓存中标识地址。

5.11

- 5.11 如5.7节所述，虚拟存储器使用一个页表来追踪虚拟地址到物理地址之间的映射。这个练习说明了当地址被访问时，页表如何更新。下表是在一个系统上所看见的虚拟地址流。假设4KiB的页，一个4项的全相联TLB，使用严格的LRU替换算法。如果必须从磁盘中取回页，那么增加下一次能取的最大页数：

4669, 2227, 13916, 34587, 48870, 12608, 49225

TLB

有效位	标记位	物理页号
1	11	12
1	7	4
1	3	6
0	4	9

页表

有效位	物理页/在磁盘上	有效位	物理页/在磁盘上
1	5	0	磁盘
0	磁盘	1	4
0	磁盘	0	磁盘
1	6	0	磁盘
1	9	1	3
1	11	1	12

- 5.11.1** [10] <5.7> 根据给出的地址流, 以及 TLB、页表的初始状态, 请给出系统的最终状态。对于每次访问, 请列出是否在 TLB 中命中, 是否在页表中命中或是发生缺页。
- 5.11.2** [15] <5.7> 重复练习题 5.11.1, 但是这次使用 16KiB 的页来代替 4KiB 的页。使用更大的页的好处有哪些? 缺点又有哪些?
- 5.11.3** [15] <5.4, 5.7> 如果使用两路组相联的 TLB, 请指出 TLB 中最终的内容。如果 TLB 是直接映射的, 同样指出 TLB 中最终的内容。讨论使用 TLB 来获得高性能的重要性。如果没有 TLB, 那么如何处理虚拟存储器访问?

有一些参数对页表整个大小会有影响。下面就列出一些关键的页表参数。

虚拟地址位数	页的大小	页表项的大小
32 位	8KiB	4 字节

- 5.11.4** [5] <5.7> 已知上表中的参数, 一个系统用了一半的内存来运行 5 个应用程序, 计算该系统使用的页表总大小。
- 5.11.5** [10] <5.7> 已知上表中的参数, 一个系统用了一半的内存来运行 5 个应用程序, 给定一个两级的有 256 项的页表, 计算该系统使用的页表总大小。假设主页表中每项是 6 字节, 计算所需的最小和最大的内存容量。
- 5.11.6** [10] <5.7> 一名 cache 设计者要将一个 4KiB 的虚拟索引、物理标记的 cache 的容量增大, 已知页的大小在上表中列出, 那么能否构建一个 16KiB 的直接映射 cache, 假设块大小为 2 个字? 设计者如何增加 cache 的数据大小?

答案:

1. 首先, 由题目可知, 每页大小为 4KiB, 所以每页的页偏移为 12 位, 除去后 12 位之后得到的就是页号, 对应 TLB 表中的标记位。这些页对应的页号如下

十进制	二进制	页号
4669	0001 0010 0011 1101	1
2227	0000 1000 1011 0011	0
13946	0011 0110 0111 1010	3
34587	1000 0111 0001 1011	8
48870	1011 1110 1110 0110	11
12608	0011 0001 0100 0000	3
49255	1100 0000 0110 0111	12

当查询第一个地址时，按照页号1查询TLB，TLB缺失，需要在页表中查询，页表中的第二项（对应页号1）有效位为0，所以存储在磁盘上，需要从磁盘读入。新建一个物理页号为13的页，写入数据，将页表中的第二项的物理页号改为13。接着需要将该页的对应关系写入TLB。关于这里的TLB替换，存在问题。

- 在计组的视频中，解释说先替换有效位为0的，再替换其他的。
- 这道题采取严格的LRU算法，应替换最久没使用的项。但是这里一开始没有说明哪一项是最久没有使用的项。所以在下面的策略中选择在TLB表中选择从后往前替换。由于这道题只要求给出系统最终的状态，而在最终的TLB中这些都会被替换，所以在这里无需关心。

之后就是按照虚拟地址流继续。第二个地址对应页号0，TLB未命中，接着查询页表，发现第一项（对应页号0）有效位为1，物理地址是5，所以更新TLB即可。后面的内容不多赘述。

Address	Virtual Page	TLB H/M	TLB		
			Valid	Tag	Physical Page
4669	1	TLB miss PT hit PF	1	11	12
			1	7	4
			1	3	6
			1 (last access 0)	1	13
2227	0	TLB miss PT hit	1 (last access 1)	0	5
			1	7	4
			1	3	6
			1 (last access 0)	1	13
13916	3	TLB hit	1 (last access 1)	0	5
			1	7	4
			1 (last access 2)	3	6
			1 (last access 0)	1	13
34587	8	TLB miss PT hit PF	1 (last access 1)	0	5
			1 (last access 3)	8	14
			1 (last access 2)	3	6
			1 (last access 0)	1	13
48870	11	TLB miss PT hit	1 (last access 1)	0	5
			1 (last access 3)	8	14
			1 (last access 2)	3	6
			1 (last access 4)	11	12
12608	3	TLB hit	1 (last access 1)	0	5
			1 (last access 3)	8	14
			1 (last access 5)	3	6
			1 (last access 4)	11	12
49225	12	TLB miss PT miss	1 (last access 6)	12	15
			1 (last access 3)	8	14
			1 (last access 5)	3	6
			1 (last access 4)	11	12

Address	Virtual Page	TLB H/M	TLB		
			Valid	Tag	Physical Page
4669	0	TLB miss PT hit	1	11	12
			1	7	4
			1	3	6
			1 (last access 0)	0	5
2227	0	TLB hit	1	11	12
			1	7	4
			1	3	6
			1 (last access 1)	0	5
13916	0	TLB hit	1	11	12
			1	7	4
			1	3	6
			1 (last access 2)	0	5
34587	2	TLB miss PT hit PF	1 (last access 3)	2	13
			1	7	4
			1	3	6
			1 (last access 2)	0	5
48870	2	TLB hit	1 (last access 4)	2	13
			1	7	4
			1	3	6
			1 (last access 2)	0	5
12608	0	TLB hit	1 (last access 4)	2	13
			1	7	4
			1	3	6
			1 (last access 5)	0	5
49225	3	TLB hit	1 (last access 4)	2	13
			1	7	4
			1 (last axxess 6)	3	6
			1 (last access 5)	0	5

2.

较大的页面大小降低了TLB的未命中率，但可能导致更高的碎片化和更低的物理内存利用率。

3. 两路组相联，按照虚拟地址进行映射，规则与cache相同

Address	Virtual Page	Tag	Index	TLB H/M	TLB			
					Valid	Tag	Physical Page	Index
4669	1	0	1	TLB miss PT hit PF	1	11	12	0
					1	7	4	1
					1	3	6	0
					1 (last access 0)	0	13	1
2227	0	0	0	TLB miss PT hit	1 (last access 1)	0	5	0
					1	7	4	1
					1	3	6	0
					1 (last access 0)	0	13	1
13916	3	1	1	TLB miss PT hit	1 (last access 1)	0	5	0
					1 (last access 2)	1	6	1
					1	3	6	0
					1 (last access 0)	1	13	1
34587	8	4	0	TLB miss PT hit PF	1 (last access 1)	0	5	0
					1 (last access 2)	1	6	1
					1 (last access 3)	4	14	0
					1 (last access 0)	1	13	1
48870	11	5	1	TLB miss PT hit	1 (last access 1)	0	5	0
					1 (last access 2)	1	6	1
					1 (last access 3)	4	14	0
					1 (last access 4)	5	12	1
12608	3	1	1	TLB hit	1 (last access 1)	0	5	0
					1 (last access 5)	1	6	1
					1 (last access 3)	4	14	0
					1 (last access 4)	5	12	1
49225	12	6	0	TLB miss PT miss	1 (last access 6)	6	15	0
					1 (last access 5)	1	6	1
					1 (last access 3)	4	14	0
					1 (last access 4)	5	12	1

直接映射

Address	Virtual Page	Tag	Index	TLB H/M	TLB			
					Valid	Tag	Physical Page	Index
4669	1	0	1	TLB miss PT hit PF	1	11	12	0
					1	0	13	1
					1	3	6	2
					0	4	9	3
2227	0	0	0	TLB miss PT hit	1	0	5	0
					1	0	13	1
					1	3	6	2
					0	4	9	3
13916	3	0	3	TLB miss PT hit	1	0	5	0
					1	0	13	1
					1	3	6	2
					1	0	6	3
34587	8	2	0	TLB miss PT hit PF	1	2	14	0
					1	0	13	1
					1	3	6	2
					1	0	6	3
48870	11	2	3	TLB miss PT hit	1	2	14	0
					1	0	13	1
					1	3	6	2
					1	2	12	3
12608	3	0	3	TLB miss PT hit	1	2	14	0
					1	0	13	1
					1	3	6	2
					1	0	6	3
49225	12	3	0	TLB miss PT miss	1	3	15	0
					1	0	13	1
					1	3	6	2
					1	0	6	3

TLB位于CPU中，使得获取物理地址的速度变快。如果没有TLB，则需要先访问内存来获得物理地址，再去具体的物理地址访问内容。（随口说的，别当成标准答案）

- 一半的内存，代表32位的虚拟地址空间被占用了一半。由于页的大小为 $8KiB = 8 \times 1024B = 2^{13}B$ ，虚拟地址空间总共可以存储的数据大小为 $2^{32}B$ ，那么一半的空间是 $2^{31}B$ ，所以总共的页数为 $2^{31}/2^{13} = 2^{18}$ 个，也就是说页表中有 2^{18} 条，每一个进程都有一个页表，每个页表项的大小为4B，所以总共的页表大小为 $5 \times 2^{18} \times 4 = 5 \times 2^{20}B = 5MiB$
- 在这个两级页表中， 2^{19} 张页表被分为256个二级页表，因此每个二级页表有 $2^{19}/256 = 2048$ 个页，页表项的大小为4个字节，所以每个页表的大小为 $2048 \times 4B = 8KiB$ ，其中每张页表覆盖了 $2048 \times 8KiB = 16MiB$ 的内存空间。
如果占用一半的内存，只启用一半的页表，即一级页表只启用了128项。那么所需最小的内存容量就是二级页表的5MiB加上一级页表的 $5 \times 128 \times 6B = 3840B$
如果启用所有页表，启用全部256项一级页表，二级页表也会翻倍，所以总的内存容量为二级页表10MiB加上一级页表7680B。
- 虚拟索引，物理标记的意思是cache的索引位采用虚拟地址，标记位采用物理地址。
页的大小为8KiB，所以页内偏移地址有13位。

16KiB大小的cache，每个cache的大小为8B，所以cache块数为 $16 \times 1024 / 8 = 2048 = 2^{11}$ ，cache索引有11位。加上1位字偏移，2位字节偏移，总该需要使用14位地址，但是页内偏移地址只有13位，这样有一半的cache不会存储任何数据。

解决办法是采取二路组相联结构的cache，这样cache的索引位降低到10位，总计需要13位地址，符合要求。

七、补充

MIPS指令集简介

存储程序概念：多种类型的指令和数据均以数字形式存储于存储器中的概念，存储程序型计算机即源于此。

MIPS操作数见下表

名字	示例	注释
32个寄存器	<code>\$s0-\$s7, \$t0-\$t9, \$zero, \$a0-\$a3, \$v0-\$v1, \$gp, \$fp, \$sp, \$ra, \$at</code>	寄存器用于数据的快速存取。在MIPS中，只能对存放在寄存器中的数据执行算术操作，寄存器\$zero的值恒为0，寄存器\$at被汇编器保留，用于处理大常数
2 ³⁰ 次方个存储字	<code>Memory[0], Memory[4],, Memory[4294967292]</code>	存储器只能通过数据传输指令访问。MIPS使用字节编址，所以连续的字地址相差4。存储器用于保留数据结构、数组和溢出的寄存器

MPIS汇编指令（pdf的59页）

MIPS 汇编语言				
类别	指令	示例	含义	注释
算术	加法	<code>add \$s1, \$s2, \$s3</code>	<code>\$s1 = \$s2 + \$s3</code>	三个寄存器操作数
	减法	<code>sub \$s1, \$s2, \$s3</code>	<code>\$s1 = \$s2 - \$s3</code>	三个寄存器操作数
	立即数加法	<code>addi \$s1, \$s2, 20</code>	<code>\$s1 = \$s2 + 20</code>	用于加常数数据

MIPS 汇编语言

类别	指令	示例	含义	注释
数据传输	取字	lw \$s1,20(\$s2)	$\$s1 = \text{Memory}[\$s2 + 20]$	将一个字从内存中取到寄存器中
	存字	sw \$s1,20(\$s2)	$\text{Memory}[\$s2 + 20] = \$s1$	将一个字从寄存器中取到内存中
	取半字	lh \$s1,20(\$s2)	$\$s1 = \text{Memory}[\$s2 + 20]$	将半个字从内存中取到寄存器中
	取无符号半字	lhu \$s1,20(\$s2)	$\$s1 = \text{Memory}[\$s2 + 20]$	将半个字从内存中取到寄存器中
	存半字	sh \$s1,20(\$s2)	$\text{Memory}[\$s2 + 20] = \$s1$	将半个字从寄存器存到内存中
	取字节	lb \$s1,20(\$s2)	$\$s1 = \text{Memory}[\$s2 + 20]$	将一个字节从内存取到寄存器中
	取无符号字节	lbu \$s1,20(\$s2)	$\$s1 = \text{Memory}[\$s2 + 20]$	将一个字节从内存取到寄存器中
	存字节	sb \$s1,20(\$s2)	$\text{Memory}[\$s2 + 20] = \$s1$	将一个字节从寄存器存到内存中
	取链接字	ll \$s1,20(\$s2)	$\$s1 = \text{Memory}[\$s2 + 20]$	取字作为原子交换的前半部
	存条件字	sc \$s1,20(\$s2)	$\text{Memory}[\$s2 + 20] = \$s1;$ $\$s1 = 0 \text{ or } 1$	存字作为原子交换的后半部分
	取立即数的高位	lui \$s1,20	$\$s1 = 20 * 2^{16}$	取立即数并放到高16位
逻辑	与	and \$s1,\$s2,\$s3	$\$s1 = \$s2 \& \$s3$	三个寄存器操作数按位与
	或	or \$s1,\$s2,\$s3	$\$s1 = \$s2 \mid \$s3$	三个寄存器操作数按位或
	或非	nor \$s1,\$s2,\$s3	$\$s1 = \sim(\$s2 \mid \$s3)$	三个寄存器操作数按位或非
	立即数与	andi \$s1,\$s2,20	$\$s1 = \$s2 \& 20$	和常数按位与
	立即数或	ori \$s1,\$s2,20	$\$s1 = \$s2 \mid 20$	和常数按位或
	逻辑左移	sll \$s1,\$s2,10	$\$s1 = \$s2 \ll 10$	根据常数左移相应位
	逻辑右移	srl \$s1,\$s2,10	$\$s1 = \$s2 \gg 10$	根据常数右移相应位
条件分支	相等时跳转	beq \$s1,\$s2,25	if($\$s1 == \$s2$) go to PC + 4 + 100	相等检测; 和 PC 相关的跳转
	不相等时跳转	bne \$s1,\$s2,25	if($\$s1 \neq \$s2$) go to PC + 4 + 100	不相等检测; 和 PC 相关的跳转
	小于时置位	slt \$s1,\$s2,\$s3	if($\$s2 < \$s3$) $\$s1 = 1$; else $\$s1 = 0$	比较是否小于; beq, bne
	无符号数比较小于时置位	sltu \$s1,\$s2,\$s3	if($\$s2 < \$s3$) $\$s1 = 1$; else $\$s1 = 0$	比较是否小于无符号数
	无符号数比较小于立即数时置位	slti \$s1,\$s2,20	if($\$s2 < 20$) $\$s1 = 1$; else $\$s1 = 0$	比较是否小于常数
	无符号数比较小于无符号立即数时置位	sltiu \$s1,\$s2,20	if($\$s2 < 20$) $\$s1 = 1$; else $\$s1 = 0$	比较是否小于无符号常数
无条件跳转	跳转	j 2500	go to 10000	跳转到目标地址
	跳转至寄存器所指位置	jr \$ra	go to \$ra	用于 switch 语句, 以及过程调用
	跳转并链接	jal 2500	$\$ra = \text{PC} + 4$; go to 10000	用于过程调用

MPIS字段

op(6位)	rs(5位)	rt(5位)	rd(5位)	shamt(5位)	funct(6位)
--------	--------	--------	--------	-----------	-----------

- op: 指令的基本操作, 通常称为操作码。
- rs: 第一个源操作数寄存器。
- rt: 第二个源操作数寄存器。
- rd: 用于存放操作结果的目的寄存器。
- shamt: 位移量。
- funct: 功能。一般称为功能码, 用于指明op字段中操作的特定变式。

R型指令:

op(6位)	rs(5位)	rt(5位)	rd(5位)	shamt(5位)	funct(6位)
--------	--------	--------	--------	-----------	-----------

I型指令:

op(6位)	rs(5位)	rt(5位)	立即数或地址(16位)
--------	--------	--------	-------------

J型指令:

op(6位)	地址(26位)
--------	---------

浮点数加法

1. 对阶: 小阶向大阶看齐, 将指数小的数转化为较大指数形式
2. 相加: 将有效位相加, 带符号
3. 规格化: 将和重新规格化
4. 舍入查: 四舍五入, 检查指数是否溢出

浮点数乘法

先不考虑正负号, 做前四步

1. 指数加: 将指数相加, 注意减去偏阶
2. 再相乘: 列竖式相乘
3. 规格化: 将积重新规格化
4. 舍入查: 四舍五入, 检查指数是否溢出
5. 定符号: 确定结果的符号

浮点数运算指令

MIPS增加了32个单精度浮点寄存器\$f0~\$31，一对单精度寄存器组合成一个双精度寄存器，以偶数编号

add, sub, mul, div加s或者d表示单精度或者双精度，构成浮点运算指令

浮点比较指令有单精度c.x.s和双精度c.x.d两组，x可以取六种比较条件中的一个（eq, neq, lt, le, gt, ge）

随后，使用浮点比较为真跳转bclt或浮点比较为假跳转bcfl指令进行条件分支

单周期计算机中的控制单元

输入或输出	信号名	R 型	lw	sw	beq
输出	RegDst	1	0	X	X
	ALUSrc	0	1	1	0
	MemtoReg	0	1	X	X
	RegWrite	1	1	0	0
	MemRead	0	1	0	0
	MemWrite	0	0	1	0
	Branch	0	0	0	1
	ALUOp1	1	0	0	0
	ALUOp0	0	0	0	1

控制多选器 MUX:

RegDst: 写寄存器的目标寄存器号来自rd

ALUSrc: 第二个ALU操作数是指令低16位的符号扩展

MemtoReg: 写入存储器的数据来自于数据存储器

控制寄存器堆和存储器读写的信号:

RegWrite: 寄存器堆写使能有效

MemRead: 数据存储器读使能有效

MemWrite: 将写入数据输入端的数据写入到用地址指定存储器单元中取

决定是否转移:

Branch: 决定是否分支

ALU的两位控制信号:

ALUOP0、ALUOP1: 运算类型

cache缺失

在直接映射中会产生三种缺失:

1. 首次访问cache中没有的块必然产生的缺失，称为冷启动强制缺失
2. 由于cache容量不能容纳程序执行需要的所有块，部分块被替换后再次调入cache中，称为容量缺失

3. 多个内存块竞争映射到同一个cache块，导致仍需要使用的块被替换，称为冲突碰撞缺失
三种原因导致的缺失统称为3C模型。